4.4 因特网的路由选择协议

4.4.1 有关路由选择协议的几个基本概念

1. 理想的路由算法

- **算法必须是正确的和完整的：**沿着所指引的路由分组一定能到达。
- **算法在计算上应简单：**路由选择计算不应使网络通信量增加太多额外开销。
- **算法应能适应通信量和网络拓扑的变化：**要有自适应性。
- **算法应具有稳定性：**当网络通信量和网络拓扑相对稳定时，路由算法应收敛于一个可以接受的解，而不应使得出的路由不停变化。
- **算法应是公平的：**除少数优先级高的用户外，路由选择对所有用户平等。
- **算法应是最佳的：**应能找到最好的路由，使得平均时延最小网络吞吐量最大。不存在一种绝对的最佳路由算法，“最佳”通常只能是相对某一特定要求下的合理选择。

代价

- 在研究路由选择时，需要给每一条链路指明一定的代价。
- 这里“代价”并不是指“钱”，而是由一个或几个因素综合决定的一种度量(metric)。
  - 如链路长度、数据率、链路容量、是否要保密、传播时延等，甚至还可以是一天中某一个小时内的通信量、结点的缓存被占用的程度、链路差错率等。

最佳路由

- 不存在一种绝对的最佳路由算法。
- 所谓“最佳”只能是相对于某一种特定要求下得出的较为合理的选择而已。
- **实际的路由选择算法，应尽可能接近于理想的算法**。
- 路由选择是个非常复杂的问题
  - 它是网络中的所有结点共同协调工作的结果。
  - 路由选择的环境往往是不断变化的，而这种变化有时无法事先知道。

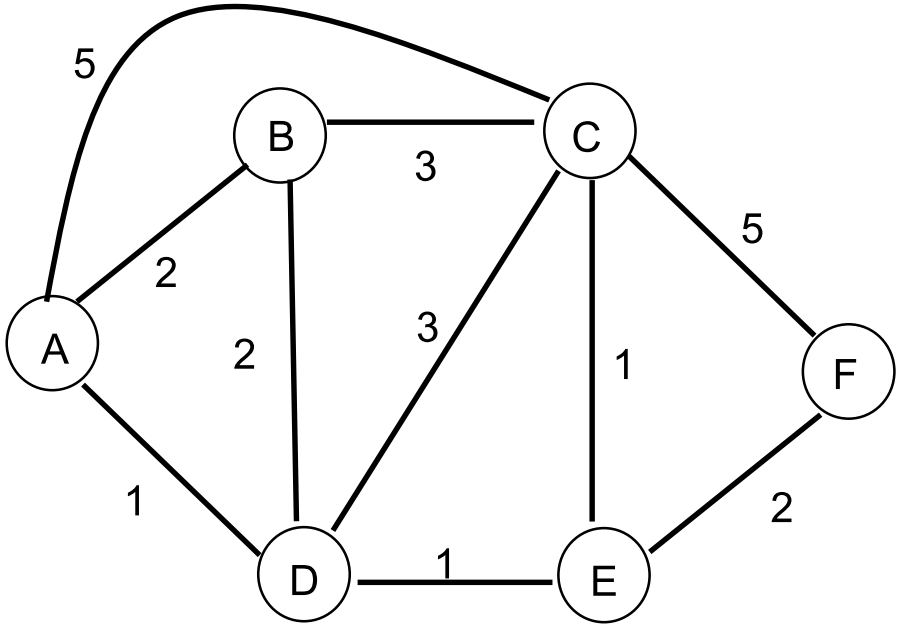
从路由算法 的自适应性考虑

- **静态路由选择策略**——即非自适应路由选择，其特点是简单和开销较小，但不能及时适应网络状态的变化。 —— Dijkstra算法
- **动态路由选择策略**——即自适应路由选择，其特点是能较好地适应网络状态的变化，但实现起来较为复杂，开销也比较大。——**距离向量路由选择算法。**
- 两种常用路由选择算法： **Dijkstra算法**和**距离向量路由选择算法。**

■ Dijkstra算法：

- 已知网络拓扑和链路“**长度**”，为网络中的结点选择最短路由。
- “**长度**”：指耗费或代价，可以是**距离**、**时延**等。

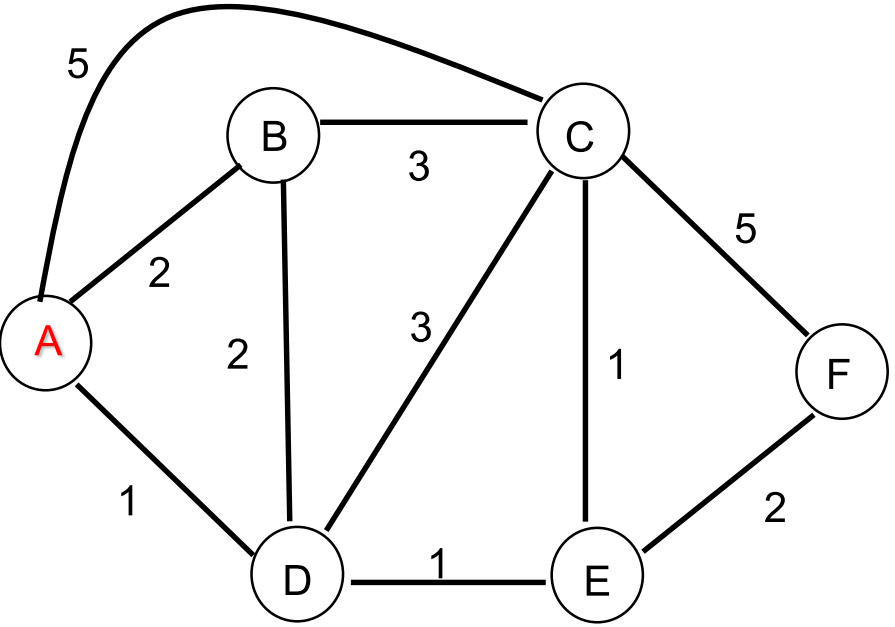
例：网络如图所示，链路上的数字表示链路长度，找出以结点A为根的最短通路树，写出A的路由表。



邻接矩阵 l(i,j)

	A	B	C	D	E	F
A	0	2	5	1	∞	∞
B	2	0	3	2	∞	∞
C	5	3	0	3	1	5
D	1	2	3	0	1	∞
E	∞	∞	1	1	0	2
F	∞	∞	5	∞	2	0

■ Dijkstra算法 ：

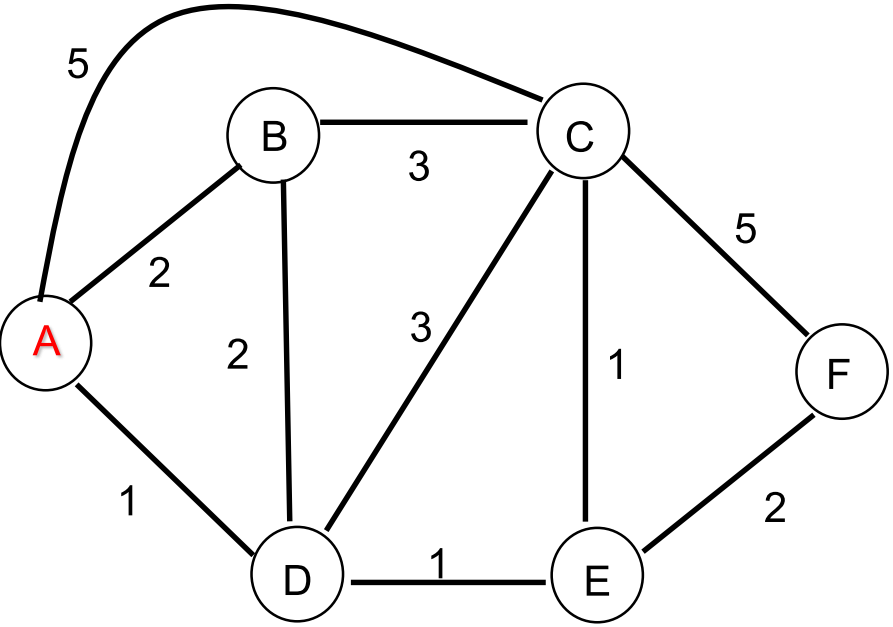


邻接矩阵 l(i,j)

	A	B	C	D	E	F
A	0	2	5	1	∞	∞
B	2	0	3	2	∞	∞
C	5	3	0	3	1	5
D	1	2	3	0	1	∞
E	∞	∞	1	1	0	2
F	∞	∞	5	∞	2	0

- 令***D(v)***为源结点(假设为结点**A**)到某个结点***v***的距离，它就是从结点A沿某一路径到结点***v***的所有链路的长度之和。再令***l(i,j)***为相邻结点*i*和*j*之间的距离。

■ Dijkstra算法 ：



邻接矩阵 l(i,j)

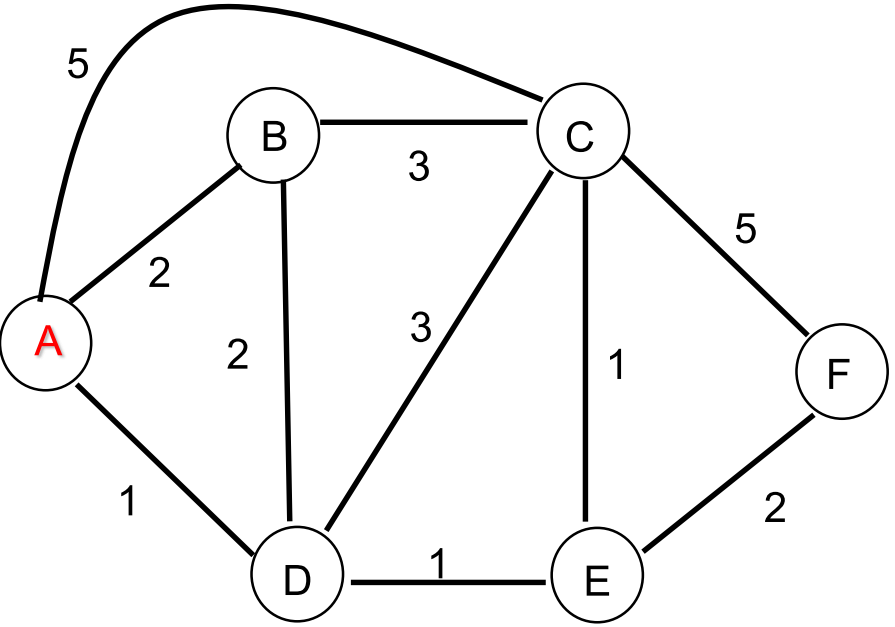
	A	B	C	D	E	F
A	0	2	5	1	∞	∞
B	2	0	3	2	∞	∞
C	5	3	0	3	1	5
D	1	2	3	0	1	∞
E	∞	∞	1	1	0	2
F	∞	∞	5	∞	2	0

(1) 初始化

令N表示网络结点的集合。先令$N = \{A\}$。对所有不在N中的结点v，写出：

$$D(v) = \begin{cases} l(A,v) & \text{若结点 } v \text{ 与结点 A 直接相连} \\ \infty & \text{若结点 } v \text{ 与结点 A 不直接相连} \end{cases}$$

■ Dijkstra算法 ：



邻接矩阵 l(i,j)

	A	B	C	D	E	F
A	0	2	5	1	∞	∞
B	2	0	3	2	∞	∞
C	5	3	0	3	1	5
D	1	2	3	0	1	∞
E	∞	∞	1	1	0	2
F	∞	∞	5	∞	2	0

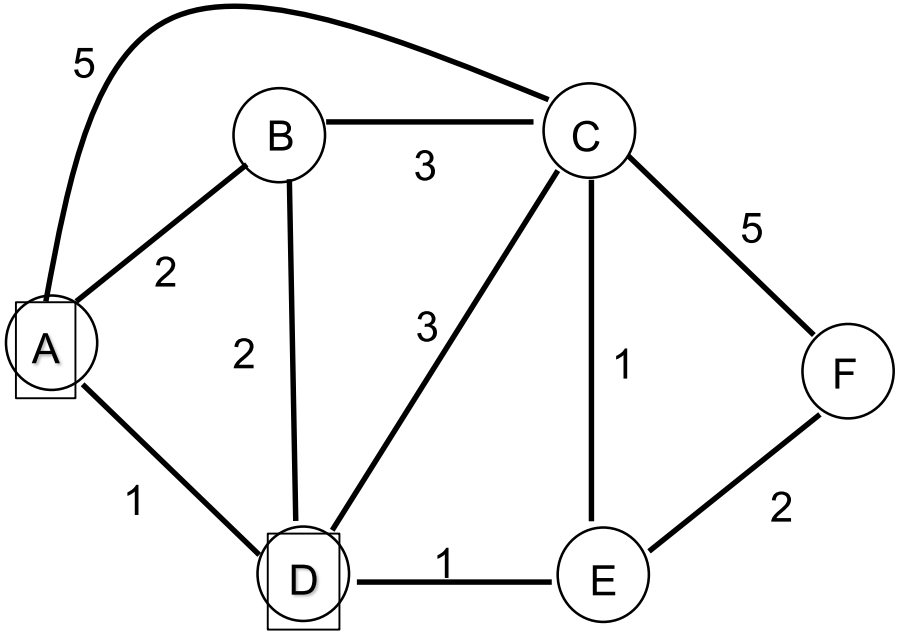
(2) 寻找一个不在N中的结点w，其$D(w)$值为最小。把w加入到N中 $N = \{A, D\}$ 。然后对所有不在N中的结点v，用$[D(v), D(w) + l(w, v)]$中的较小的值去更新原有的$D(v)$值，即：

$$D(v) \leftarrow \text{Min}[D(v), D(w) + l(w, v)]$$

(3) 重复步骤(2)，直到所有的网络结点都在N中为止。

Dijkstra算法：

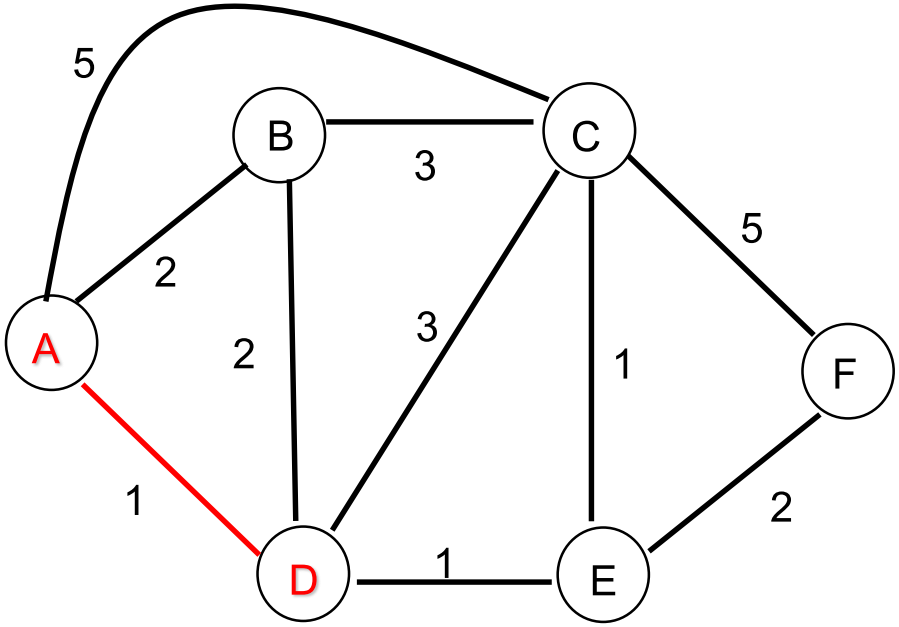
目的结点	最短路由	距离
B	A→B	2
C	A→C	5
D	A→D	1
E	A→E	∞
F	A→ F	∞



步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞

Dijkstra算法：

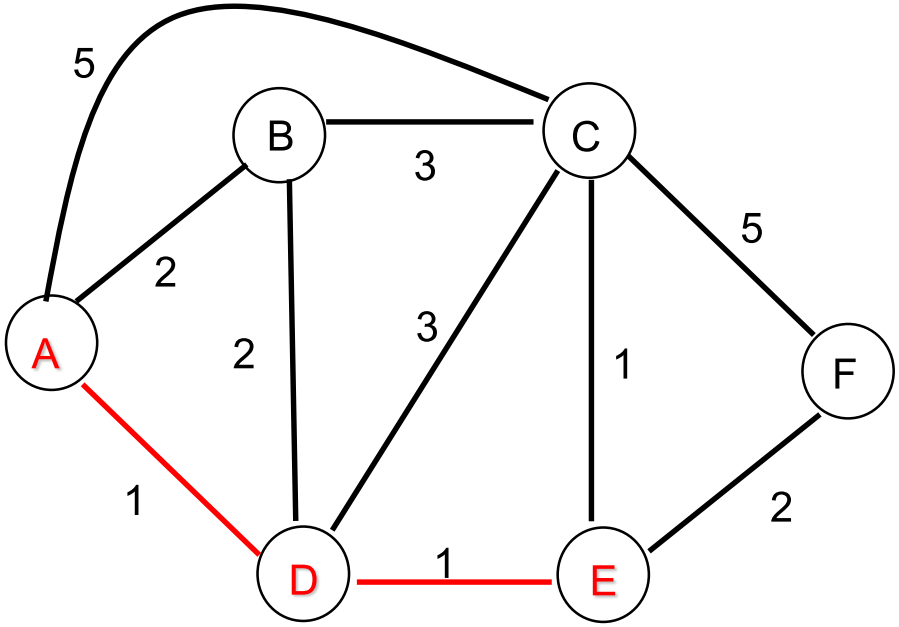
目的结点	最短路由	距离
B	A→B	2
C	A→D→C	4
D	A→D	1
E	A→D→E	2
F	A→F	∞



步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞
1	{A, D}	2	4	1	2	∞

Dijkstra算法：

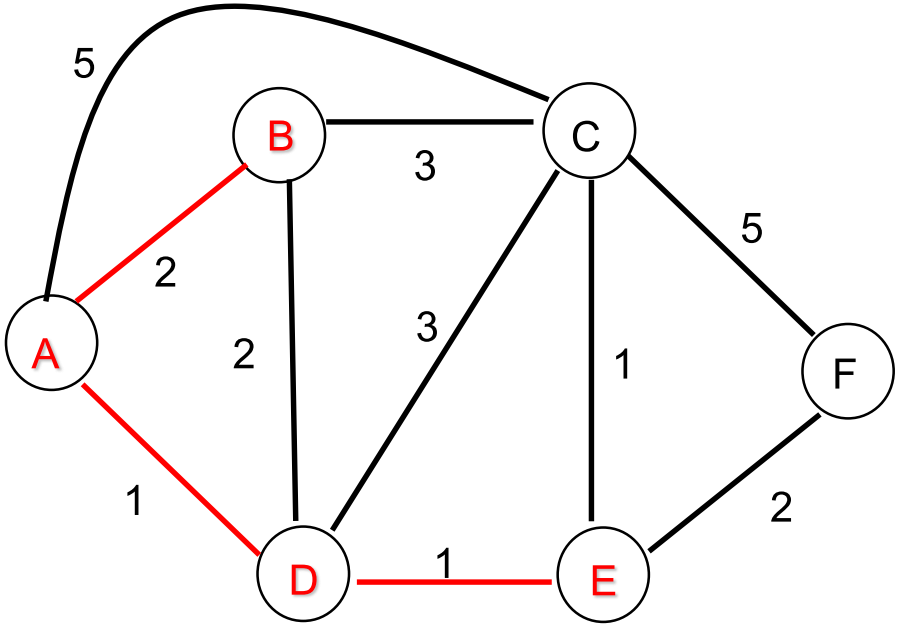
目的结点	最短路由	距离
B	A→B	2
C	A→D→E→C	3
D	A→D	1
E	A→D→E	2
F	A→D→E→F	4



步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞
1	{A, D}	2	4	1	2	∞
2	{A, D, E}	2	3	1	2	4

Dijkstra算法 ：

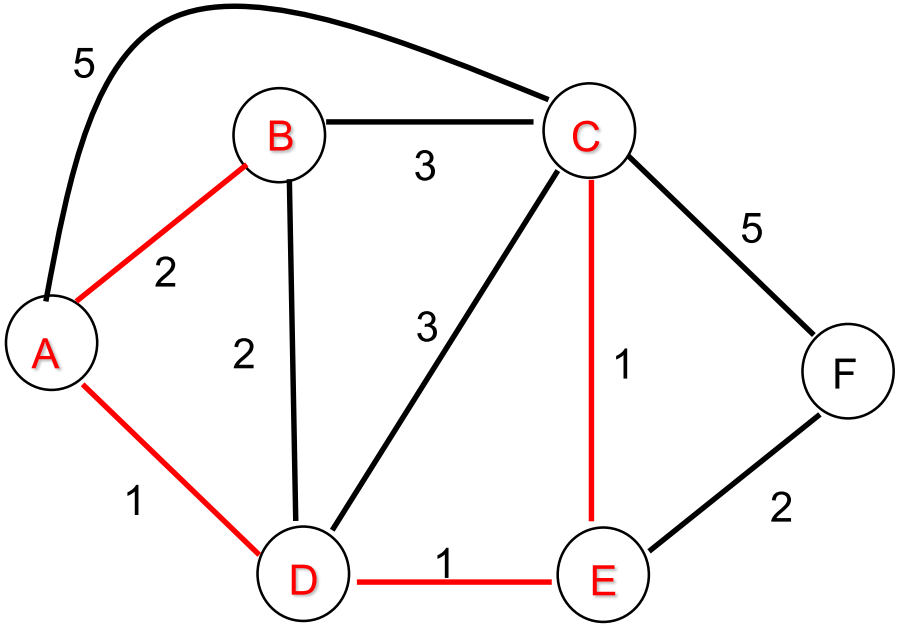
目的结点	最短路由	距离
B	A→B	2
C	A→D→E→C	3
D	A→D	1
E	A→D→E	2
F	A→D→E→F	4



步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞
1	{A, D}	2	**4**	1	**2**	∞
2	{A, D, E}	2	**3**	1	**2**	**4**
3	{A, D, E, B}	2	**3**	1	2	4

Dijkstra算法：

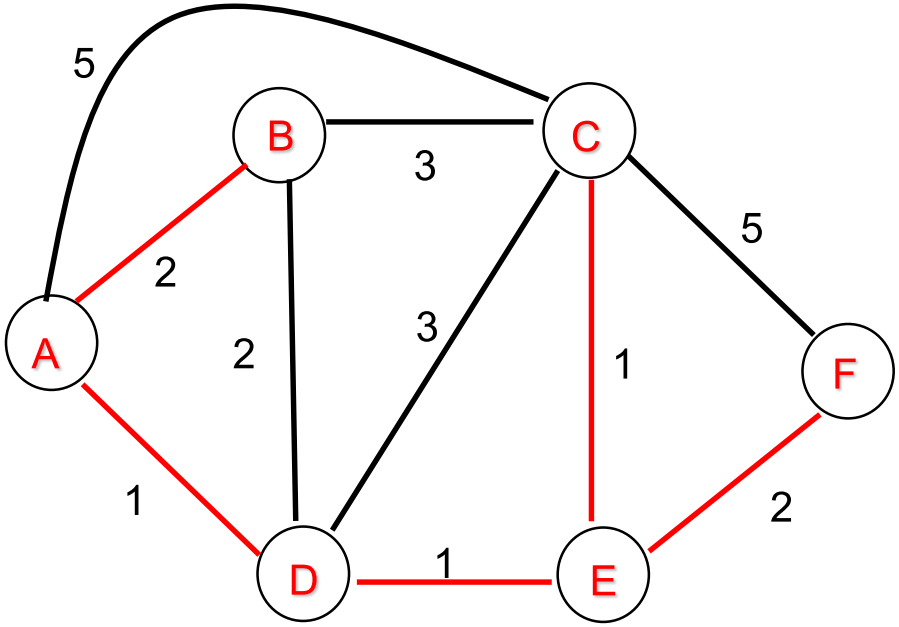
目的结点	最短路由	距离
B	A→B	2
C	A→D→E→C	3
D	A→D	1
E	A→D→E	2
F	A→D→E→F	4



步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞
1	{A, D}	2	**4**	1	**2**	∞
2	{A, D, E}	2	**3**	1	**2**	**4**
3	{A, D, E,B}	2	3	1	2	4
4	{A, D, E,B,C}	2	3	1	2	4

Dijkstra算法：

目的结点	最短路由	距离
B	A→B	2
C	A→D→E→C	3
D	A→D	1
E	A→D→E	2
F	A→D→E→F	4



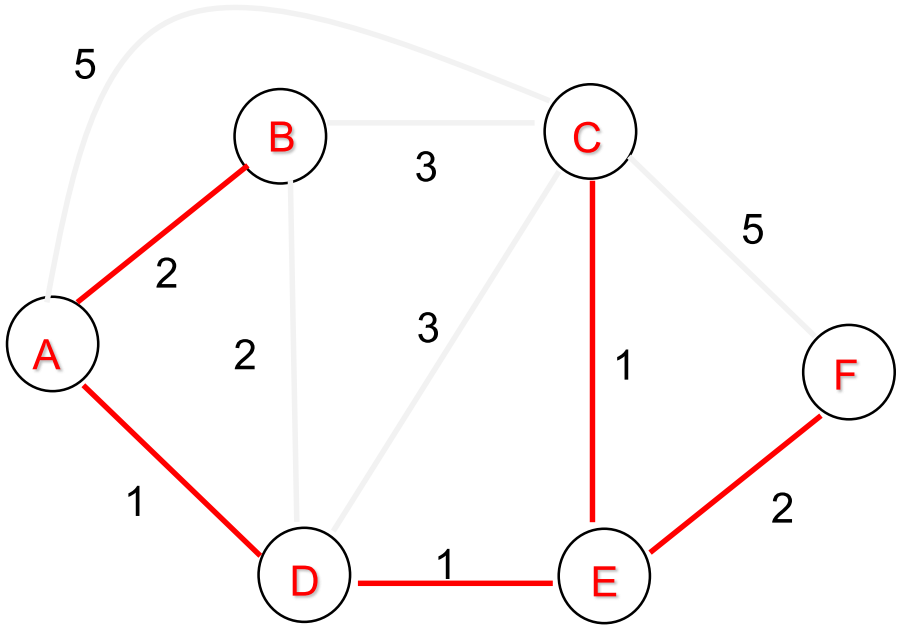
步骤	N	D(B)	D(C)	D(D)	D(E)	D(F)
初始化	{A}	2	5	1	∞	∞
1	{A, D}	2	**4**	1	**2**	∞
2	{A, D, E}	2	**3**	1	**2**	**4**
3	{A, D, E,B}	2	3	1	2	4
4	{A, D, E,B,C}	2	3	1	2	4
5	{A, D, E,B,C,F}	2	3	1	2	4

Dijkstra算法：

结点A为根的最短通路树

结点A的路由表

目的结点	下一跳结点	距离
B	B	2
C	D	3
D	D	1
E	D	2
F	D	4



结点A到其他结点的最短路由

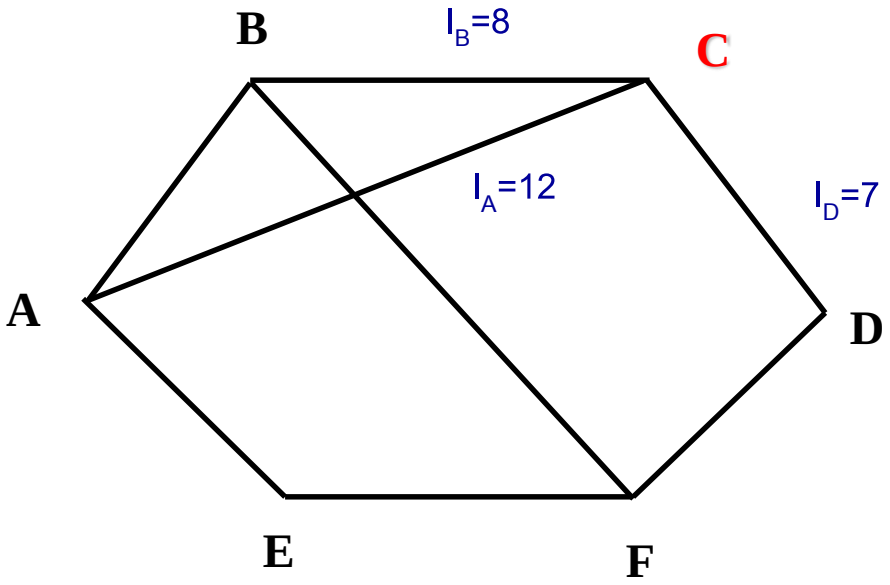
目的结点	最短路由	距离
B	A→B	2
C	A→D→E→C	3
D	A→D	1
E	A→D→E	2
F	A→D→E→F	4

距离向量路由选择：

- 每个路由器（结点）维护一张路由表（向量），表中给出到每个目的地址的最短“距离”和路线，通过与相邻路由器（结点）交换信息来更新路由表的信息。
- 每一个路由器都指明了到其他路由器（结点）的下一跳路由器（结点）及其最短距离。通过不断搜索“下一跳路由器”得到到达目标站点的最短路由。
- 度量的标准可以是中继段数、估计的延迟时间、排队的分组数、剩余的容量等。

■ 距离向量路由选择：

例：结点C收到的相邻结点发来的延迟信息，相邻结点P到每个目的结点L的延时为PL(mS)，当前结点C到相邻结点P的延迟为I(mS)，**求结点C的最新路由表**。



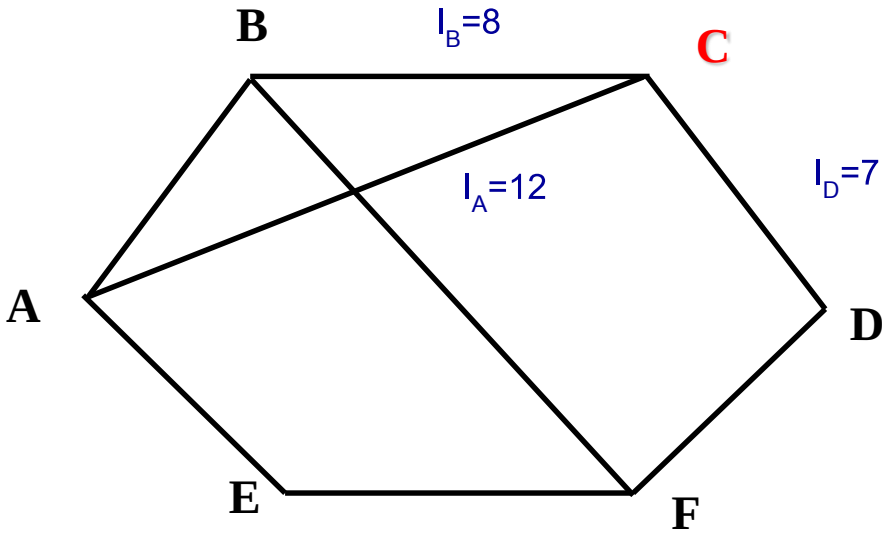
结点C收到的相邻结点发来的延迟信息(ms)

PL:

	B	A	D
A	3	0	31
B	0	2	20
C	9	11	4
D	19	30	0
E	25	20	27
F	13	22	6
	I=8	I=12	I=7

每个路由器将他的延迟信息发送给相邻路由器，也从相邻路估计到由器收到一份类似的清单，以更新自己的路由表

■ 距离向量路由选择：



结点C收到的相邻结点发来的延迟信息(ms)

PL:

	B	A	D
A	3	0	31
B	0	2	20
C	9	11	4
D	19	30	0
E	25	20	27
F	13	22	6
	I=8	I=12	I=7

结点C的路由表

目的结点	下一跳结点	延迟
A	B	11
B	B	8
D	D	7
E	A	32
F	D	13

目的结点	下一跳结点		
	B	A	D
A	**11**	12	38
B	**8**	14	27
D	27	42	**7**
E	33	**32**	34
F	21	34	**13**

- 当前结点C经过相邻结点P（A、B或C）到达其他结点L的延迟：

  I+PL（ms）

因特网的路由选择协议

- 因特网采用分层次的路由选择协议。
- 因特网的规模非常大。如果让所有的路由器知道所有的网络应怎样到达，则这种路由表将非常大，处理起来也太花时间。而所有这些路由器之间交换路由信息所需的带宽就会使因特网的通信链路饱和。
- 许多单位不愿意外界了解自己单位网络的布局细节和本部门所采用的路由选择协议（这属于本部门内部的事情），但同时还希望连接到因特网上。

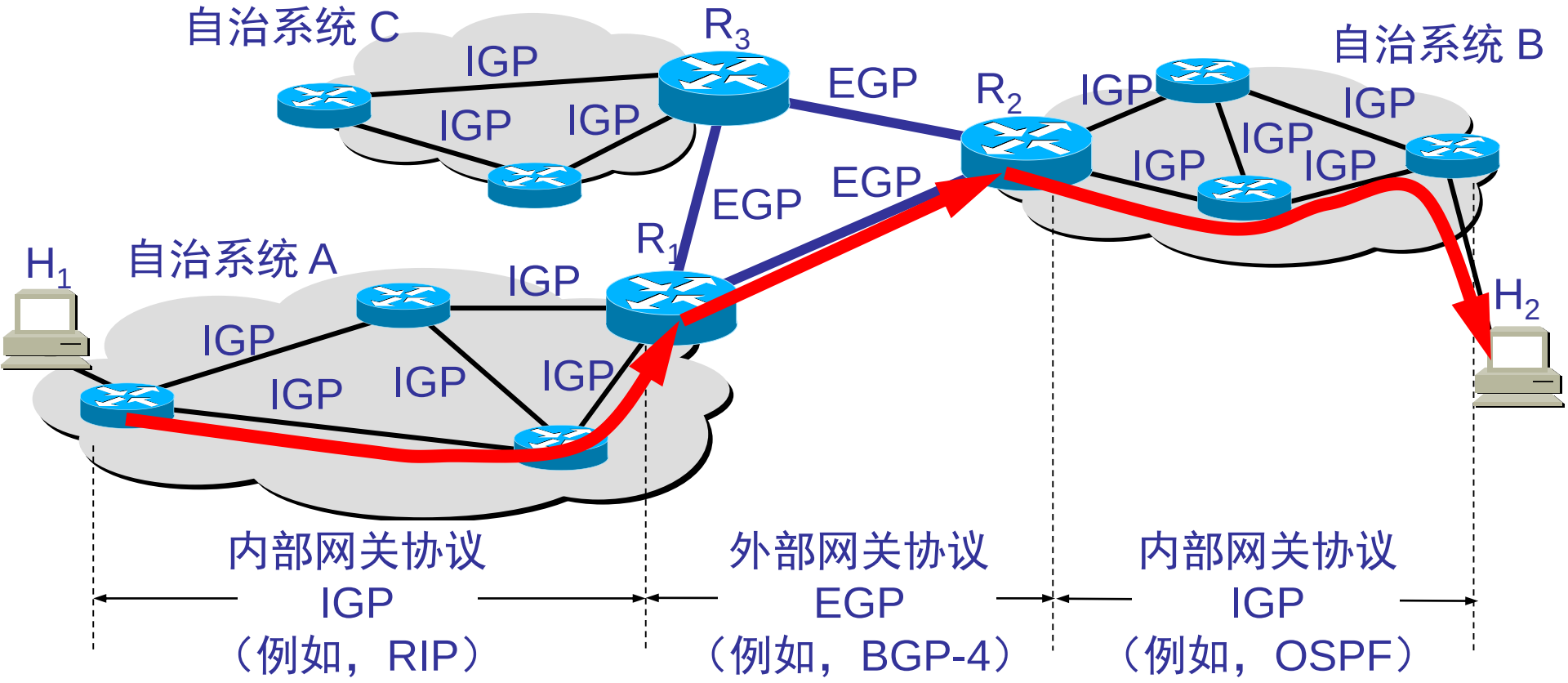
自治系统(autonomous system)

- 因特网将整个互联网划分为许多较小的自治系统 AS。
- 一个自治系统是一个互联网，其最重要的特点就是**自治系统有权自主地决定在本系统内应采用何种路由选择协议**。
- 一个自治系统内的所有网络都属于一个行政单位(例如、一个公司、一所大学、政府的一个部门、等等)来管辖。
- 一个自治系统的所有路由器在本自治系统内都必须是连通的。

因特网有两大类路由选择协议

- 内部网关协议 IGP (Interior Gateway Protocol)
  - 即在一个自治系统内部使用的路由选择协议。
  - 目前这类路由选择协议使用得最多，如 RIP 和 OSPF 协议。
- 外部网关协议EGP (External Gateway Protocol)
  - 若源站和目的站处在不同的自治系统中，当数据报传到一个自治系统的边界时，就需要使用一种协议将路由选择信息传递到另一个自治系统中。这样的协议就是外部网关协议 EGP。
  - 在外部网关协议中目前使用最多的是 BGP-4。

自治系统和
内部网关协议、外部网关协议



内部网关协议——RIP
(Routing Information Protocol)

1. 工作原理

- 路由信息协议 RIP 是内部网关协议 IGP中最先得到广泛使用的协议。
- RIP 是一种分布式的基于**距离向量**的路由选择协议。
- RIP 协议要求网络中的每一个路由器都要维护从它自己到其他每一个目的网络的距离记录。

“距离”的定义

- 从一路由器到**直接连接**的网络的距离定义为 1。
- 从一个路由器到非直接连接的网络的距离定义为所经过的路由器数加 1。
- RIP 协议中的“距离”也称为“**跳数**”(hop count)，因为每经过一个路由器，跳数就加 1。

“距离”的定义

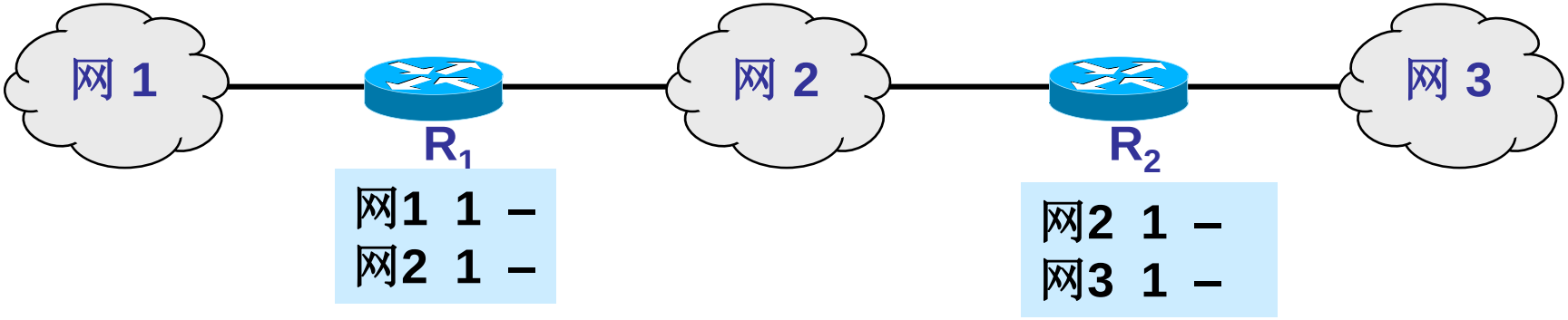
- RIP 认为一个好的路由就是它通过的路由器的数目少，即“距离短”。
- RIP 允许一条路径最多只能包含 15 个路由器。
- “距离”的最大值为16 时即相当于不可达。可见 RIP 只适用于小型互联网。
- RIP 不能在两个网络之间同时使用多条路由。RIP 选择一个具有最少路由器的路由（即最短路由），哪怕还存在另一条高速(低时延)但路由器较多的路由。

RIP 协议的三个要点

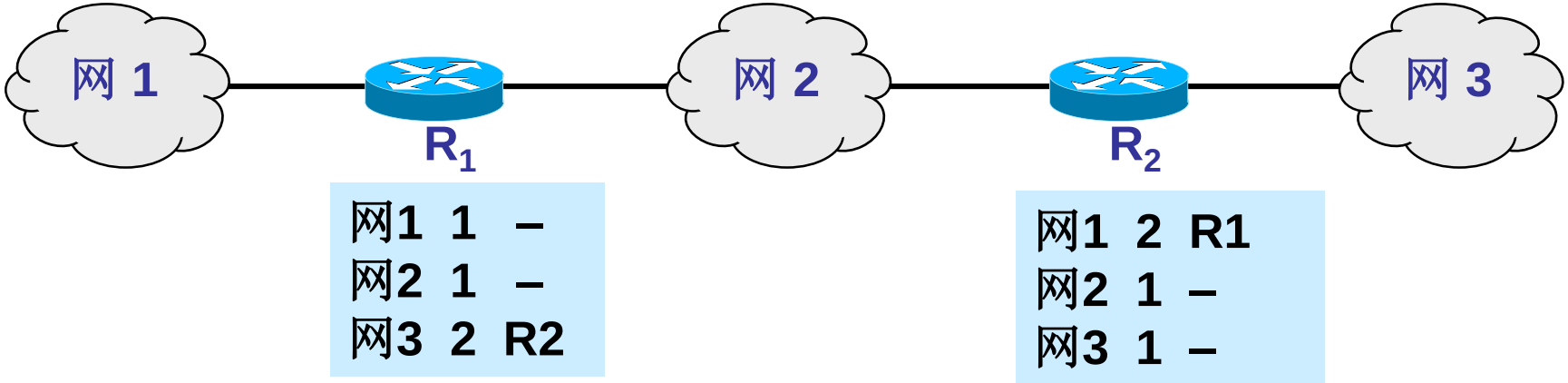
- **和哪些路由器交换信息？**——仅和相邻路由器交换信息。
- **交换什么信息？**——交换的信息是当前本路由器所知道的全部信息，即自己的路由表。
- **什么时候交换信息？**——按固定的时间间隔交换路由信息，例如，每隔 30 秒。

路由表的建立

- 路由器在刚刚开始工作时，只知道到直接连接的网络的距离（此距离定义为1）。



- 以后，每一个路由器只和数目非常有限的相邻路由器交换并更新路由信息。经过若干次更新后，所有的路由器最终都会知道到达本自治系统中任何一个网络的最短距离和下一跳路由器的地址。

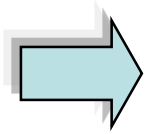


■ 距离向量算法——交换路由信息

例： 已知R6的路由表，现在收到相邻路由器R4发来的路由更新信息，试更新R6的路由表。

R4的路由表

目的网络	距离	下一跳路由
Net1	3	R1
Net2	4	R2
Net3	1	直接交付



R6的路由表

目的网络	距离	下一跳路由
Net2	3	R4
Net3	4	R5
…	…	…

收到相邻路由器R_X的一个RIP报文：

(1) 修改此RIP报文中的所有项目：把“下一跳”字段中的地址都改为R_X，并把所有的“距离”字段的值加1。

目的网络	距离	下一跳路由
Net1	4	R4
Net2	5	R4
Net3	2	R4

距离向量算法——交换路由信息

修改后的RIP报文

目的网络	距离	下一跳路由
Net1	4	R4
Net2	5	R4
Net3	2	R4



R6的路由表

目的网络	距离	下一跳路由
Net2	3	R4
Net3	4	R5
…	…	…

(2) 将修改后的RIP报文中的每一项与本路由表逐一比较：

若项目中的目的网络不在本路由表中，则把该项目加到路由表中。

R6的路由表（1）

目的网络	距离	下一跳路由
Net1	4	R4
Net2	3	R4
Net3	4	R5
…	…	…

距离向量算法——交换路由信息

目的网络	距离	下一跳路由
Net1	4	R4
Net2	5	R4
Net3	2	R4



R6的路由表（1）

目的网络	距离	下一跳路由
Net1	4	R4
Net2	3	R4
Net3	4	R5
…	…	…

否则（目的网络在本路由表中），

若下一跳字段给出的路由器地址与本路由器是同样的，则用收到的项目替换原路由表中的项目。

R6的路由表（2）

目的网络	距离	下一跳路由
Net1	4	R4
Net2	5	R4
Net3	2	R4
…	…	…

否则（下一跳路由器地址与本路由器不同）

若收到项目中的距离小于路由表中的距离，则进行更新，否则，什么也不做。

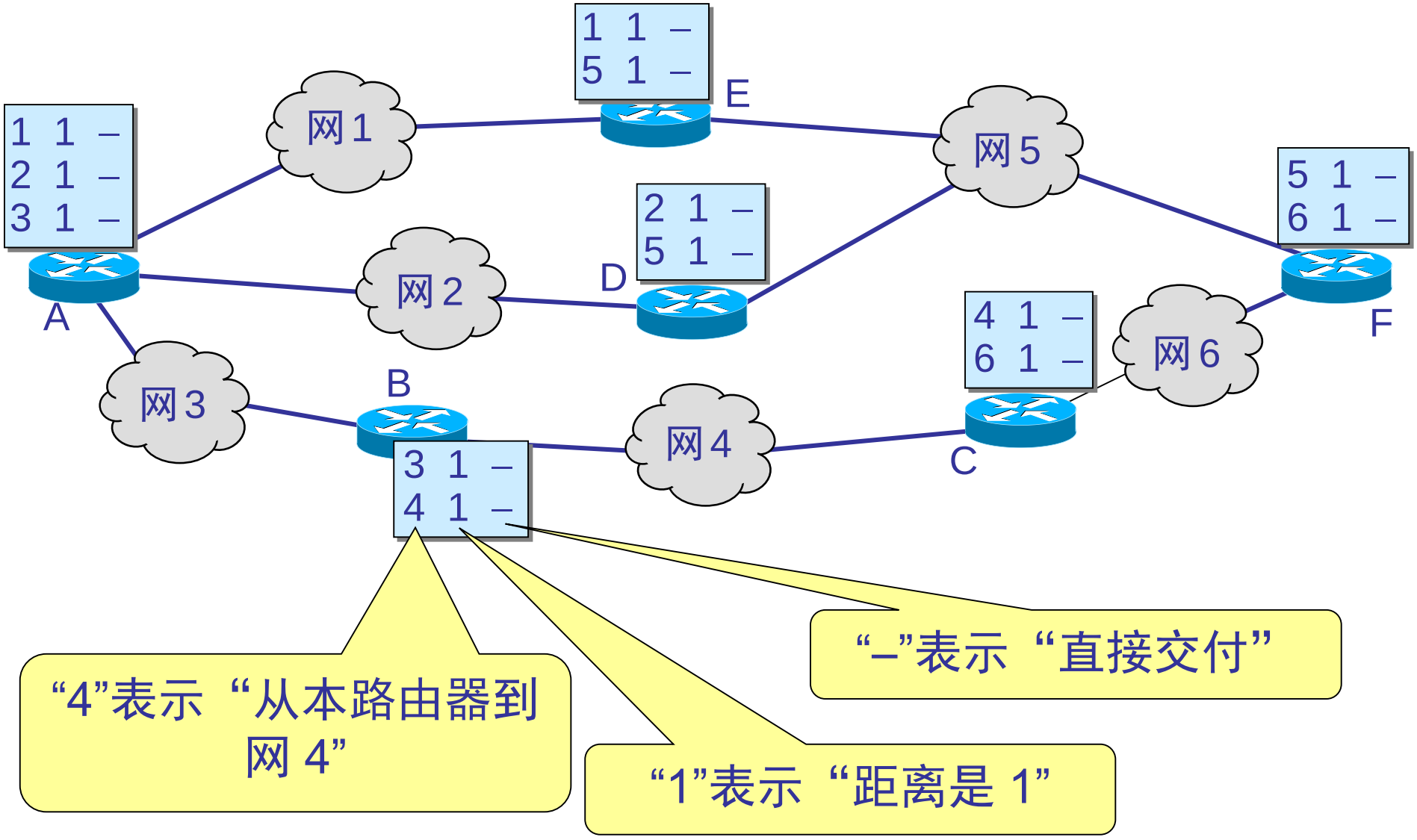
■ 距离向量算法——交换路由信息

(3) 若3分钟还没有收到相邻路由器的更新路由表，则把此路由器记为不可达路由器，即将距离置为16（距离为16表示不可达）。

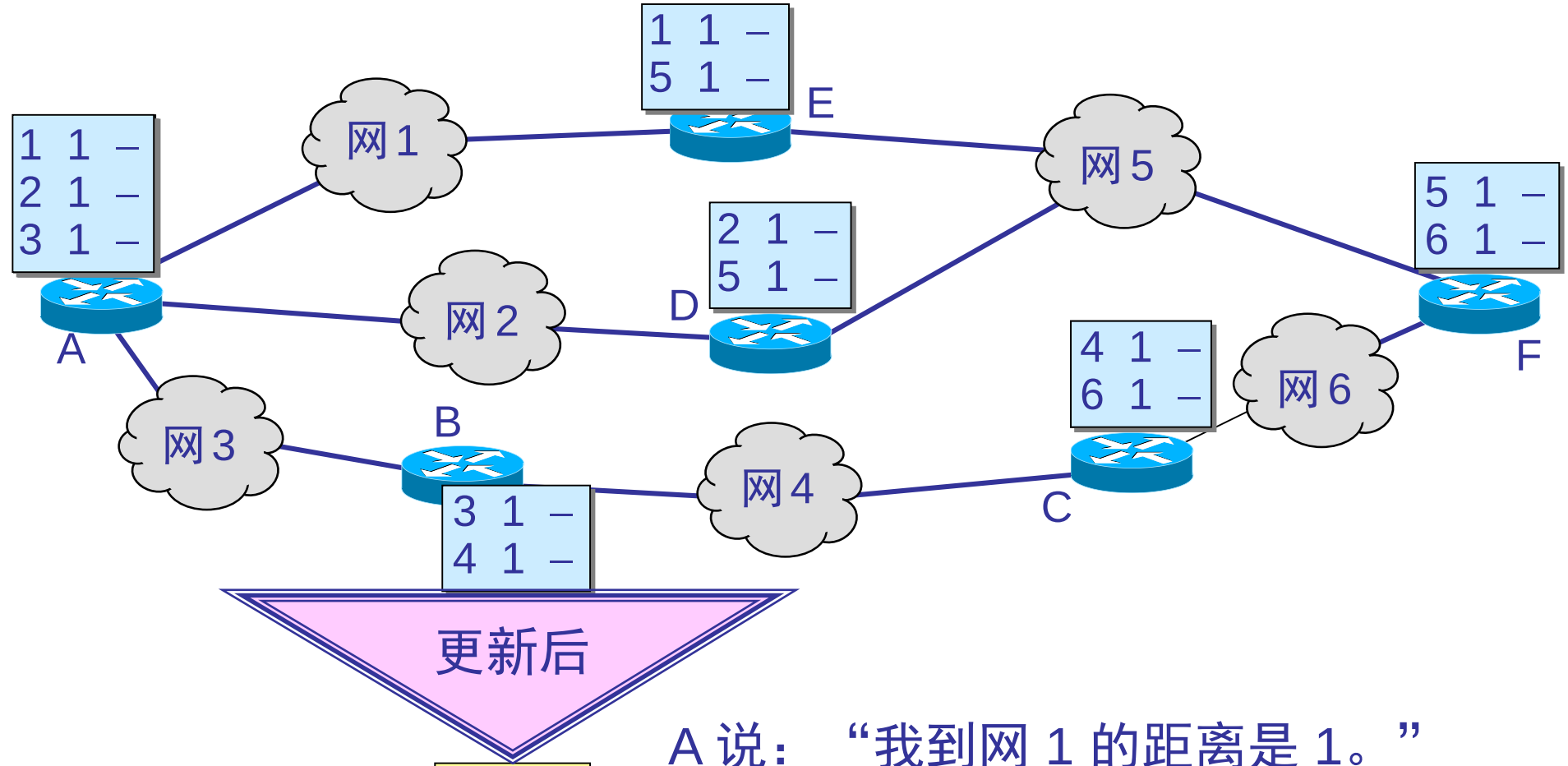
(4) 返回。

- 通过以上距离向量算法，RIP协议让互联网中的所有路由器都和自己的相邻路由器不断交换路由信息，并不断更新其路由表，使得从每一个路由器到每一个目的网络的路由都是最短的（即跳数最少）。
- 虽然所有的路由器最终都拥有了整个自治系统的全局路由信息，但由于每一个路由器的位置不同，它们的路由表当然也应当是不同的。

一开始，各路由表只有到相邻路由器的信息



路由器 B 收到相邻路由器 A 和 C 的路由表

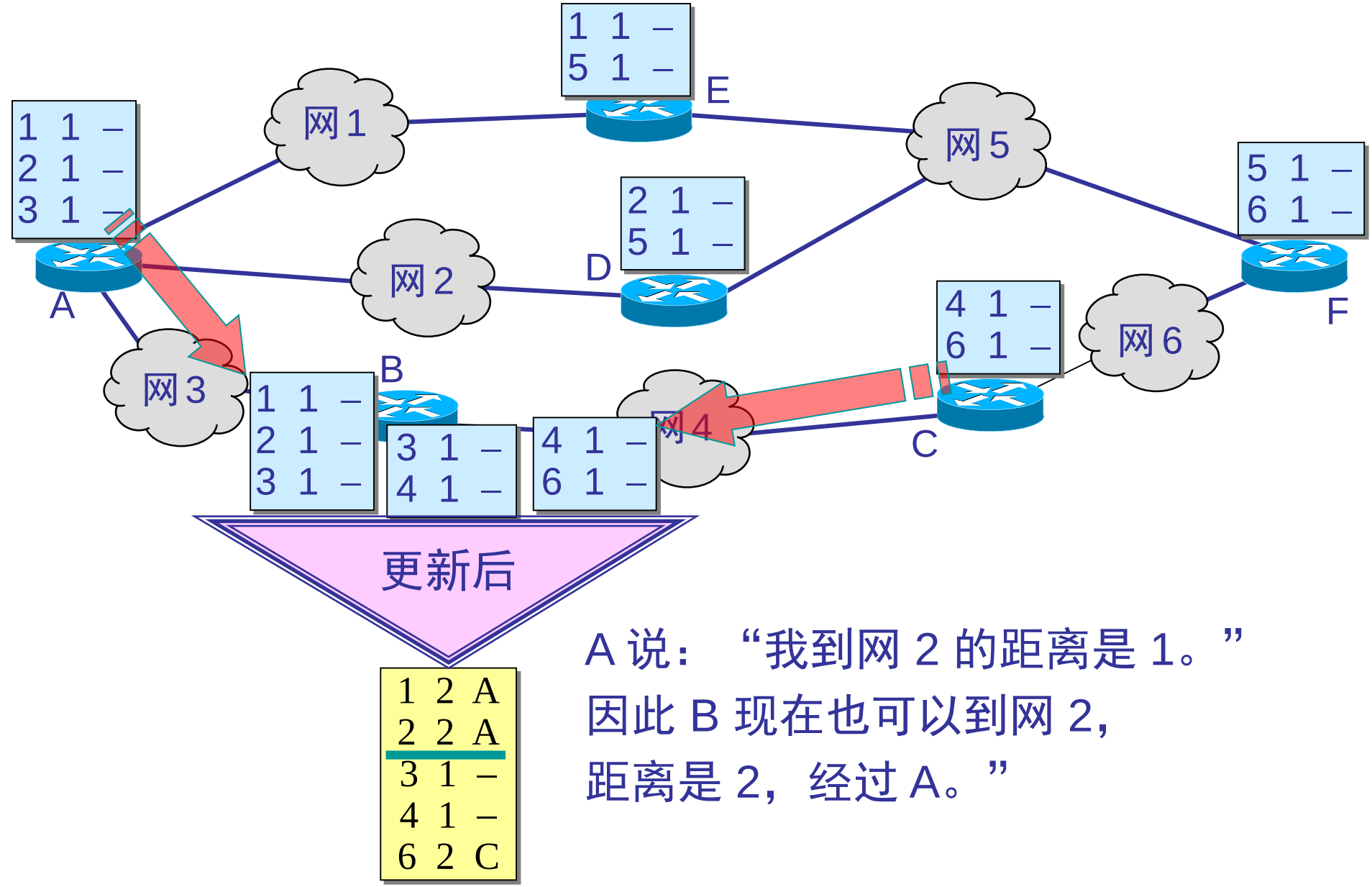


A 说：“我到网 1 的距离是 1。”

因此 B 现在也可以到网 1，

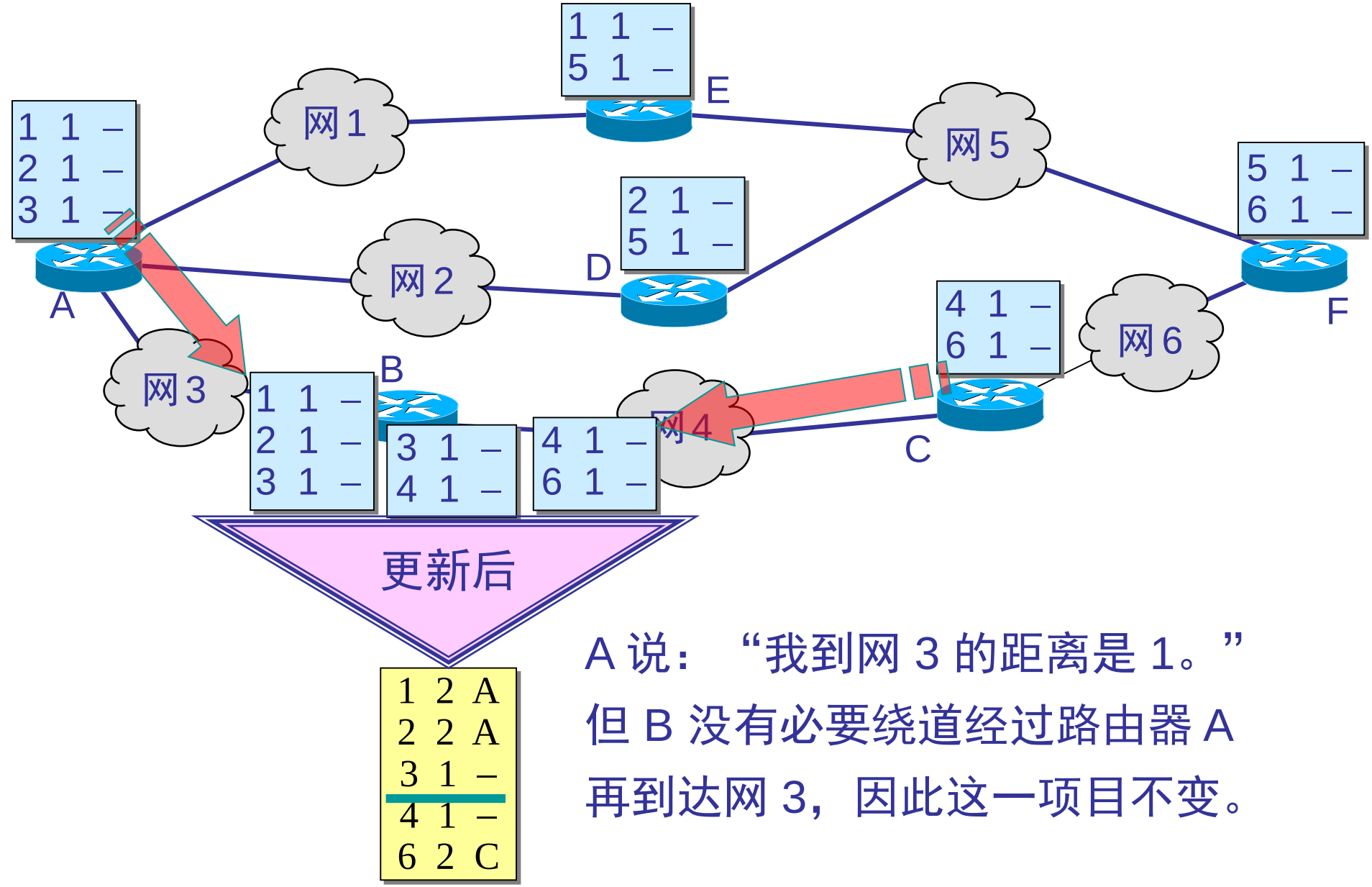
距离是 2，经过 A。”

路由器 B 收到相邻路由器 A 和 C 的路由表



A 说：“我到网 2 的距离是 1。”
因此 B 现在也可以到网 2，
距离是 2，经过 A。”

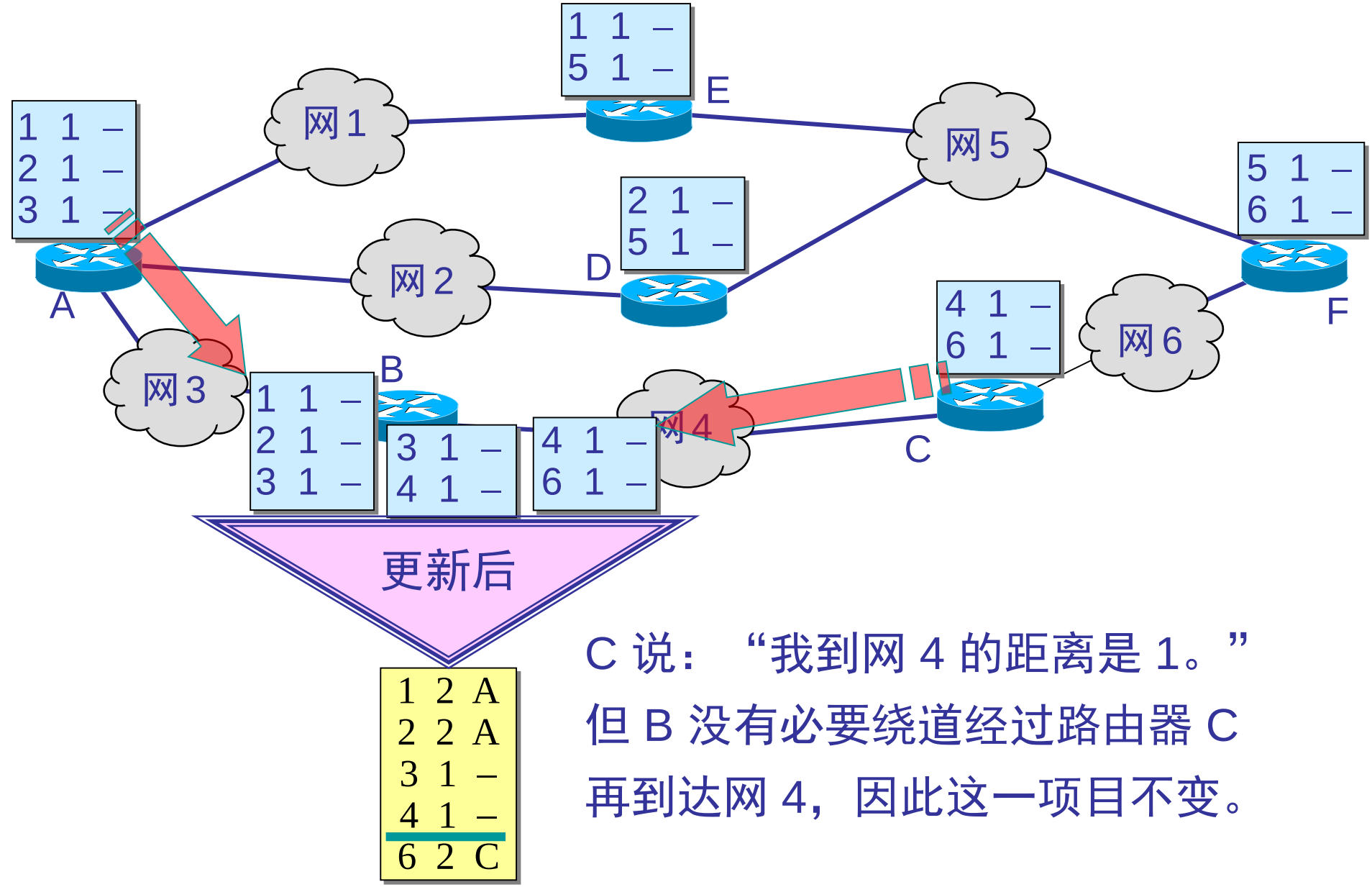
路由器 B 收到相邻路由器 A 和 C 的路由表



A 说：“我到网 3 的距离是 1。”

但 B 没有必要绕道经过路由器 A 再到达网 3，因此这一项目不变。

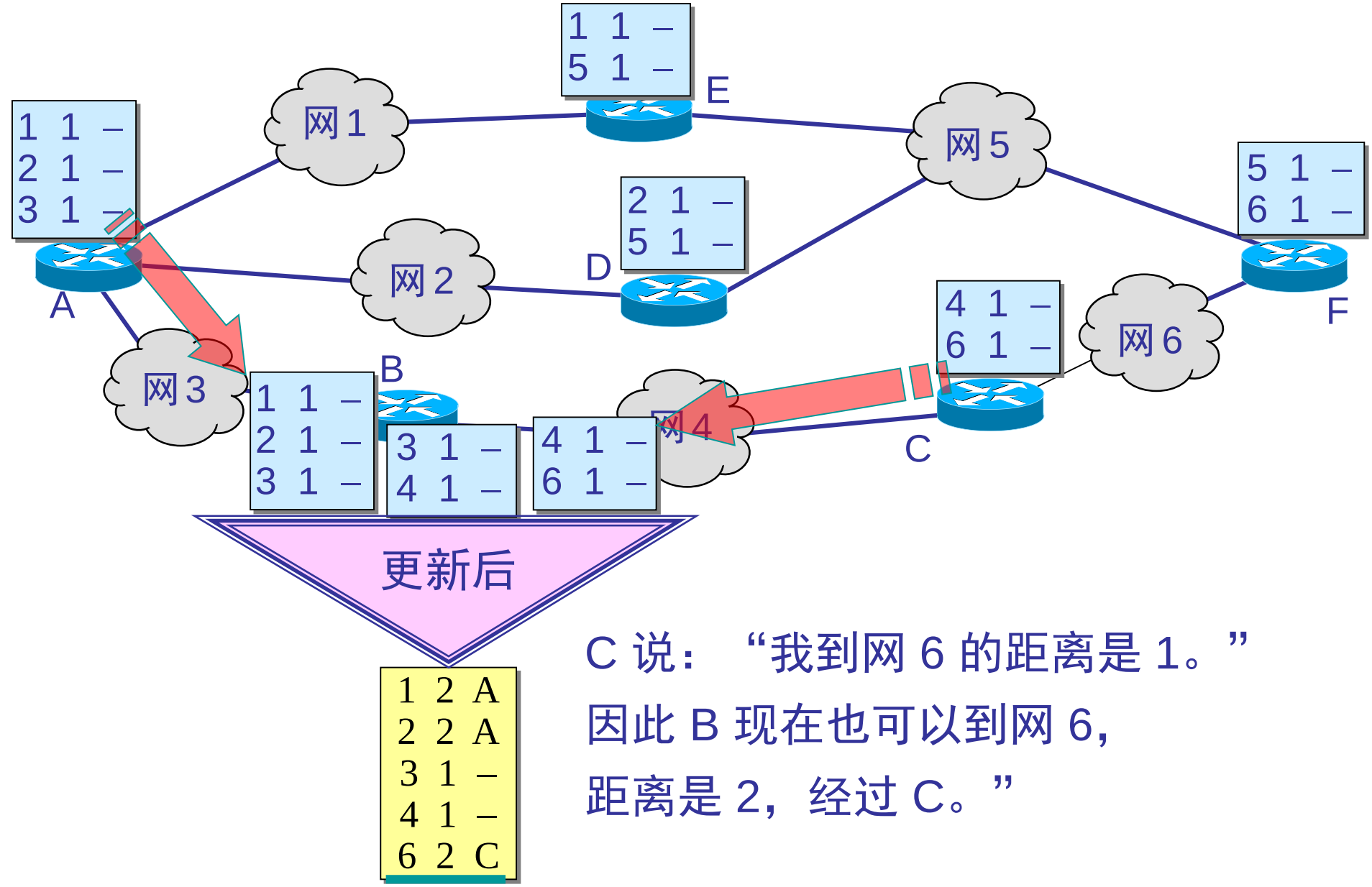
路由器 B 收到相邻路由器 A 和 C 的路由表



C 说：“我到网 4 的距离是 1。”

但 B 没有必要绕道经过路由器 C 再到达网 4，因此这一项目不变。

路由器 B 收到相邻路由器 A 和 C 的路由表

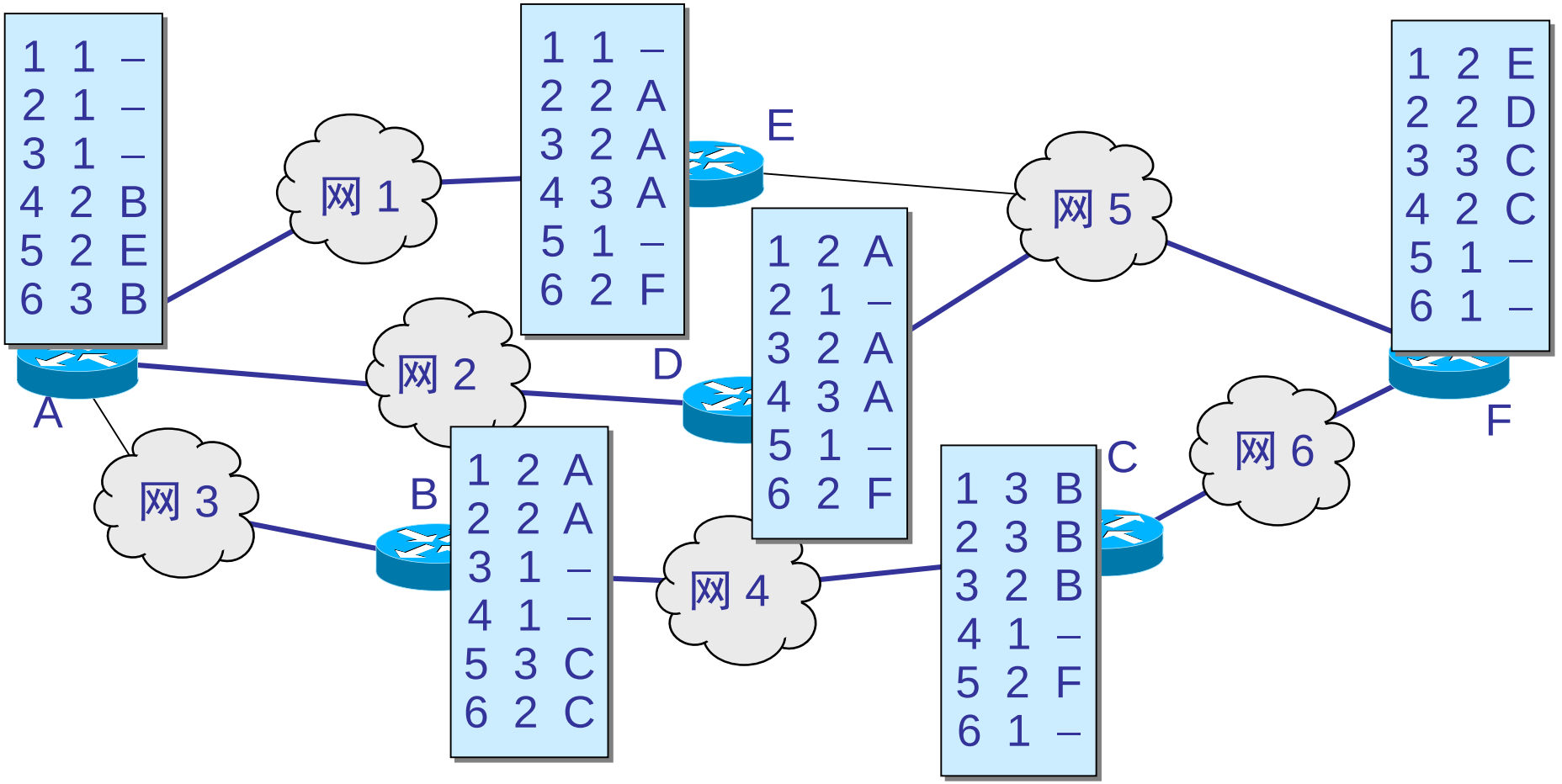


C 说：“我到网 6 的距离是 1。”

因此 B 现在也可以到网 6，

距离是 2，经过 C。”

最终所有的路由器的路由表都更新了



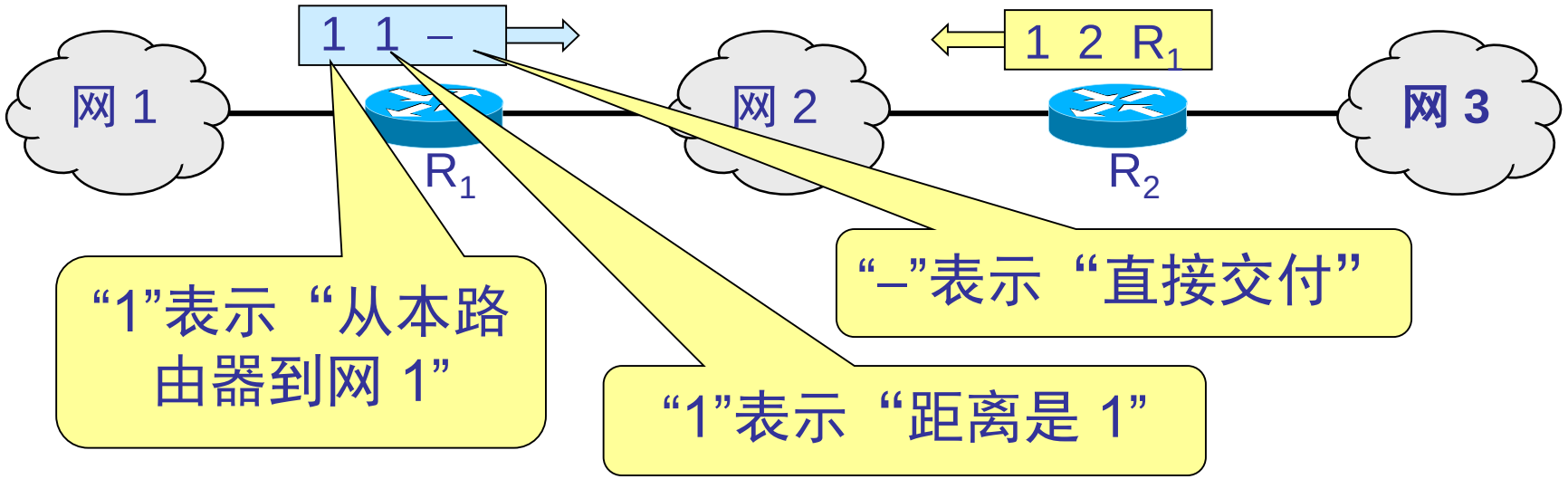
RIP 协议的位置

- RIP 协议使用运输层的用户数据报 UDP进行传送（使用 UDP 的端口 520）。
- 因此 RIP 协议的位置应当在应用层。但转发 IP 数据报的过程是在网络层完成的。

RIP 协议的优缺点

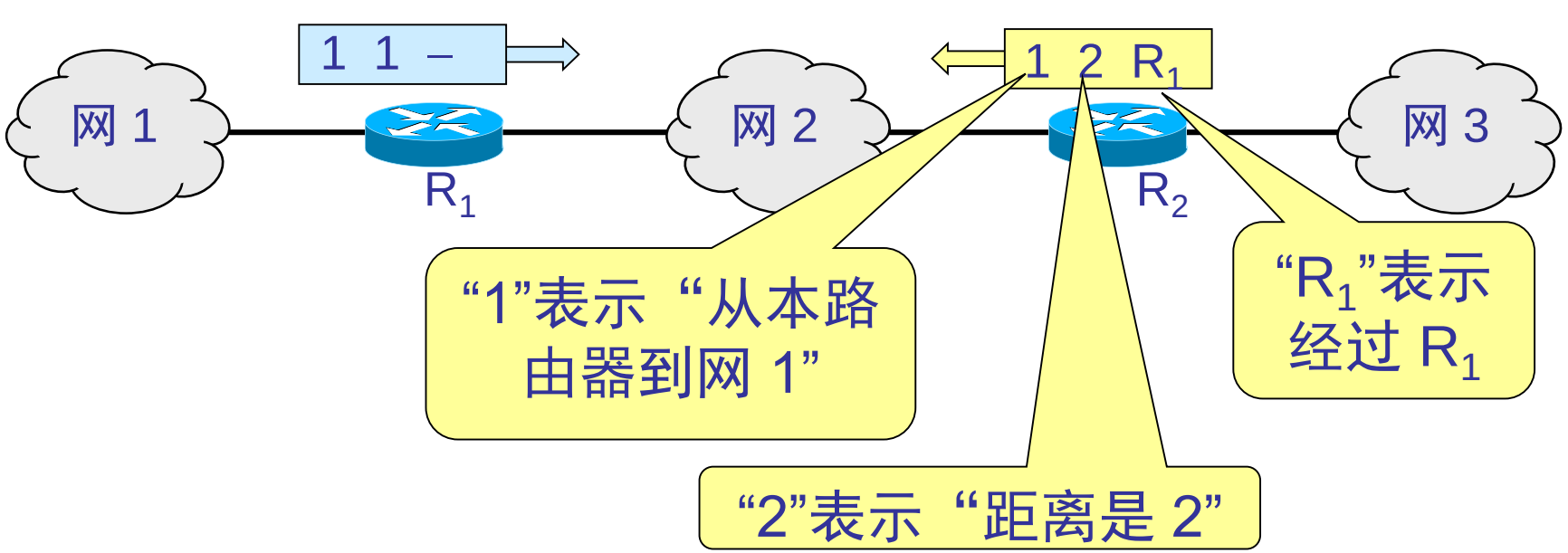
- RIP 存在的一个问题是当网络出现故障时，要经过比较长的时间才能将此信息传送到所有的路由器。
- RIP 协议最大的优点就是实现简单，开销较小。
- RIP 限制了网络的规模，它能使用的最大距离为 15（16 表示不可达）。
- 路由器之间交换的路由信息是路由器中的完整路由表，因而随着网络规模的扩大，开销也就增加。

正常情况

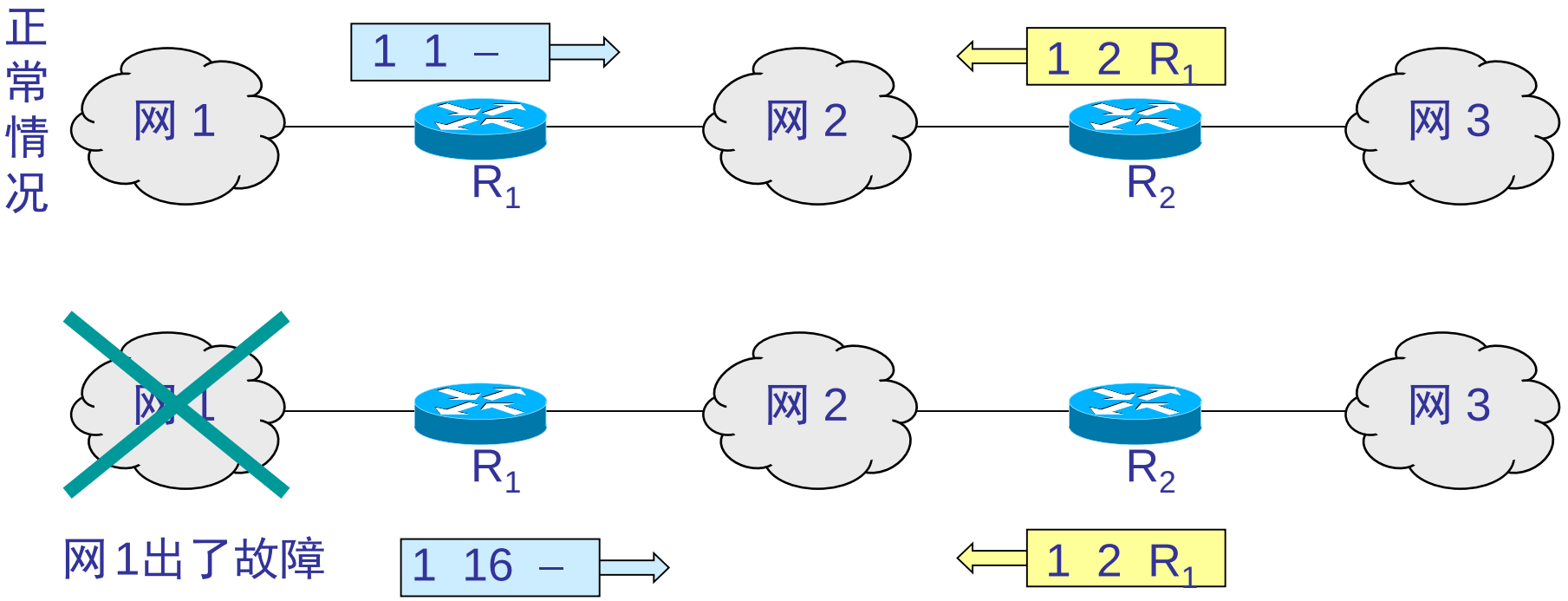


R_1 说：“我到网 1 的距离是 1，是直接交付。”

正常情况

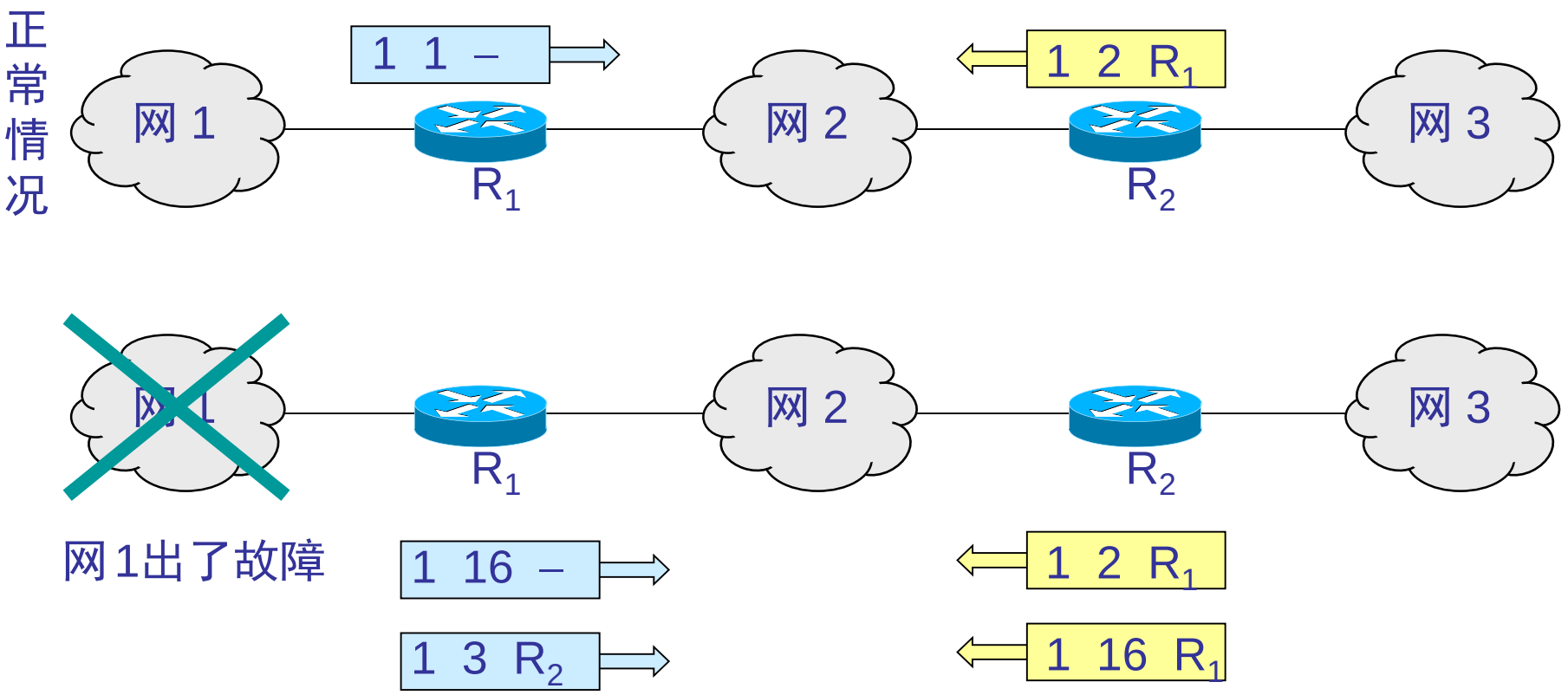


R_2 说：“我到网 1 的距离是 2，是经过 R_1。”



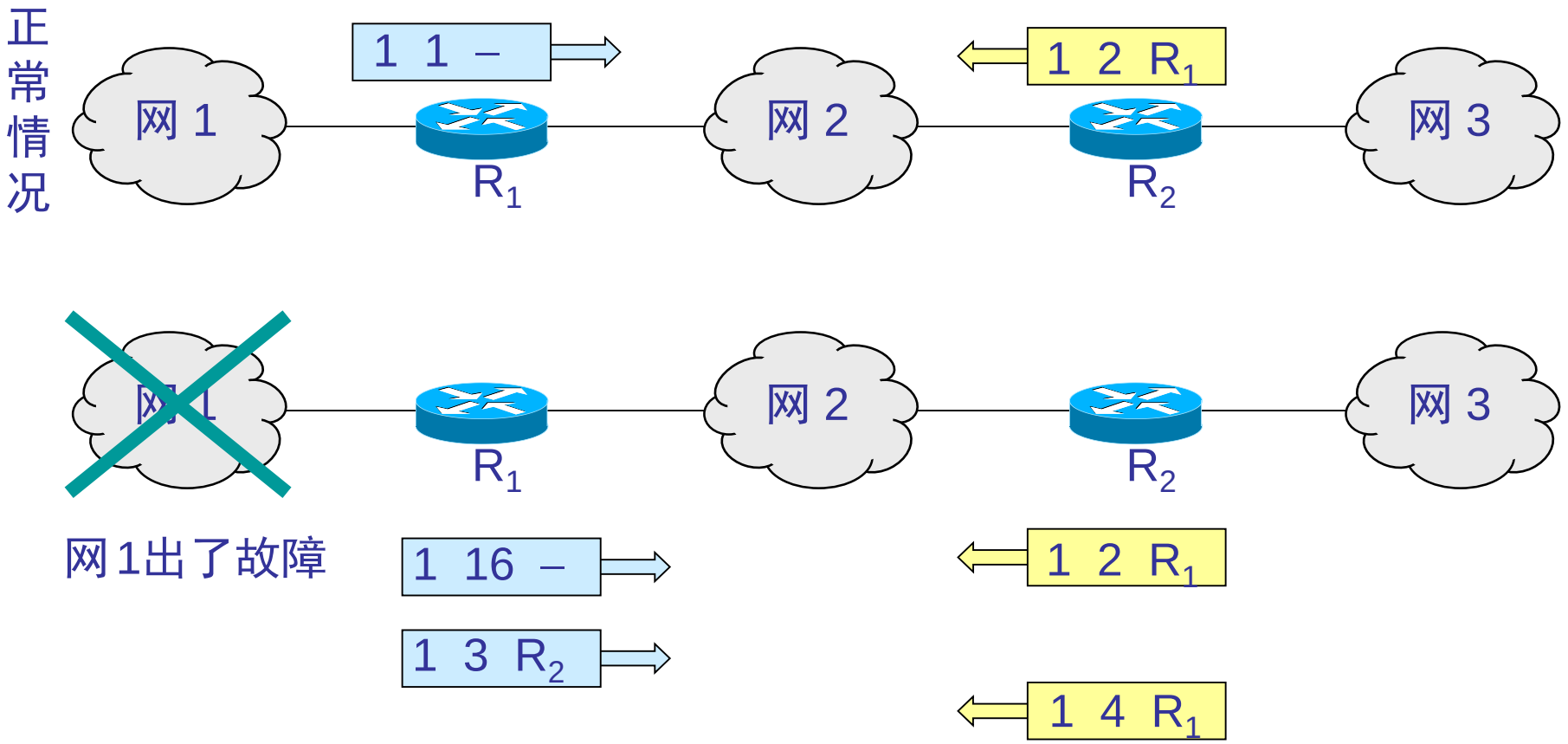
R_1 说：“我到网 1 的距离是 16 （表示无法到达），是直接交付。”

但 R_2 在收到 R_1 的更新报文之前，还发送原来的报文，因为这时 R_2 并不知道 R_1 出了故障。



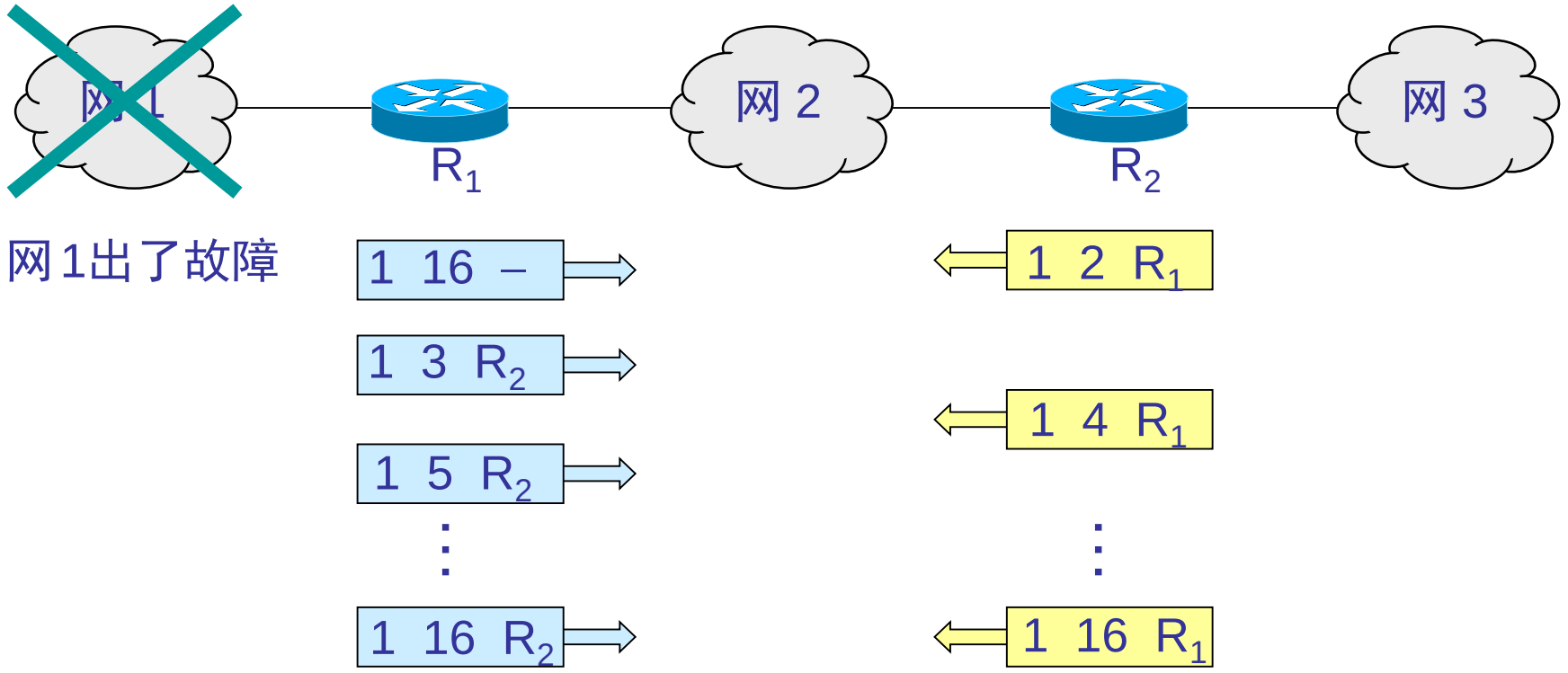
R_1 收到 R_2 的更新报文后，误认为可经过 R_2 到达网1，于是更新自己的路由表，并将此更新信息发送给 R_2。说：“我到网 1 的距离是 3，下一跳经过 R_2”。

同样，R_2 收到 R_1 的更新报文后，将路由信息更新为1 16 R_1，于是更新自己的路由表，将此路由信息发给R_1。说：“我到网 1 的距离是 16，下一跳经过 R_1”。然后将此更新信息发送给 R_2。



R_2 以后又更新自己的路由表为“1, 4, R_1”，表明 “我到网 1 距离是 4，下一跳经过 R_1”。

这就是好消息传播得快，而坏消息传播得慢。网络出故障的传播时间往往需要较长的时间(例如数分钟)。这是 RIP 的一个主要缺点。



这样不断更新下去，直到 R_1 和 R_2 到网 1 的距离都增大到 16 时，R_1 和 R_2 才知道网 1 是不可达的。

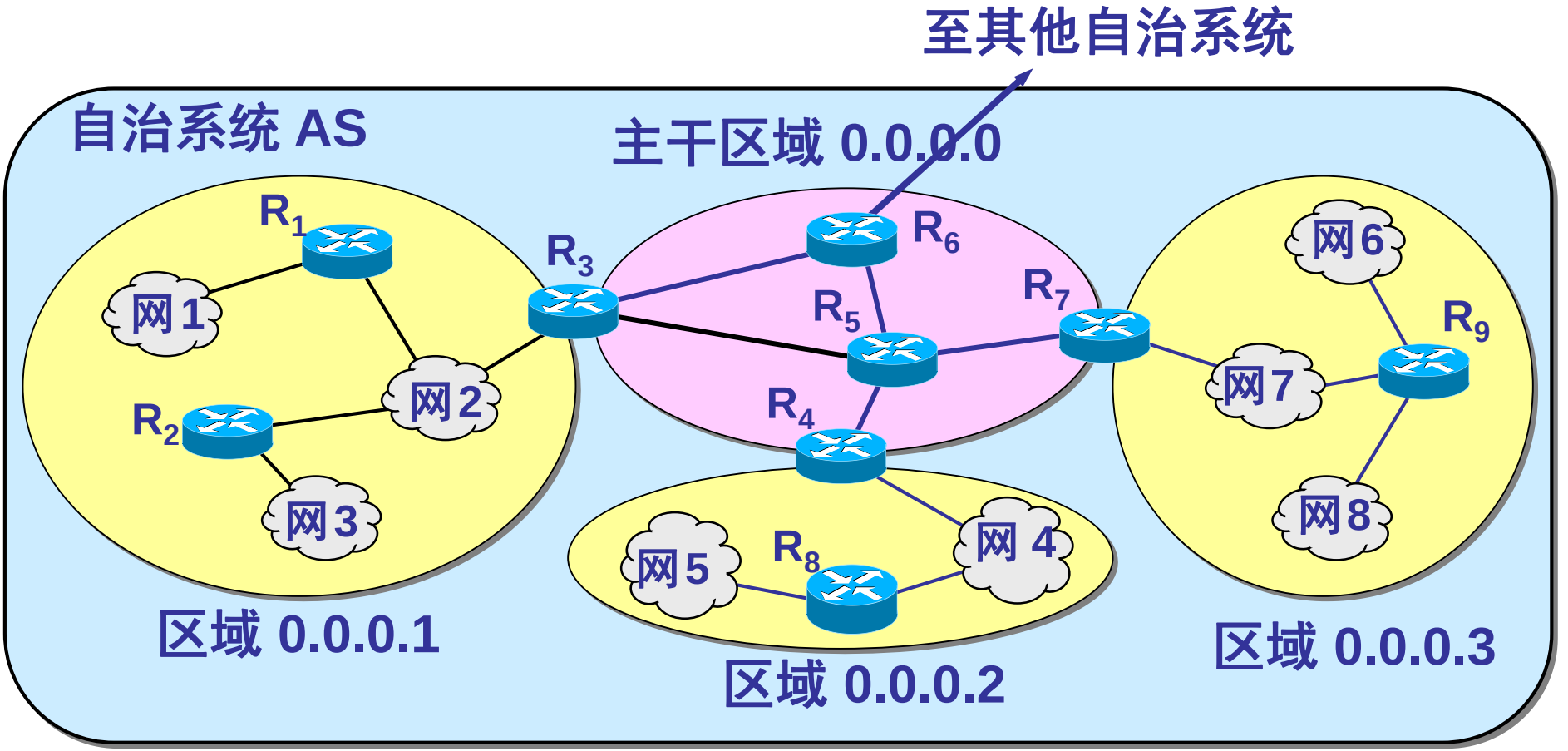
内部网关协议 ——OSPF
(Open Shortest Path First)

1. OSPF 协议的基本特点
   - “开放”表明 OSPF 协议不是受某一家厂商控制，而是公开发表的。
   - “最短路径优先”是因为使用了 Dijkstra 提出的最短路径算法SPF。
   - OSPF 只是一个协议的名字，它并不表示其他的路由选择协议不是“最短路径优先”。
   - 是分布式的**链路状态协议**。而不是RIP的**距离向量协议**。

三个要点

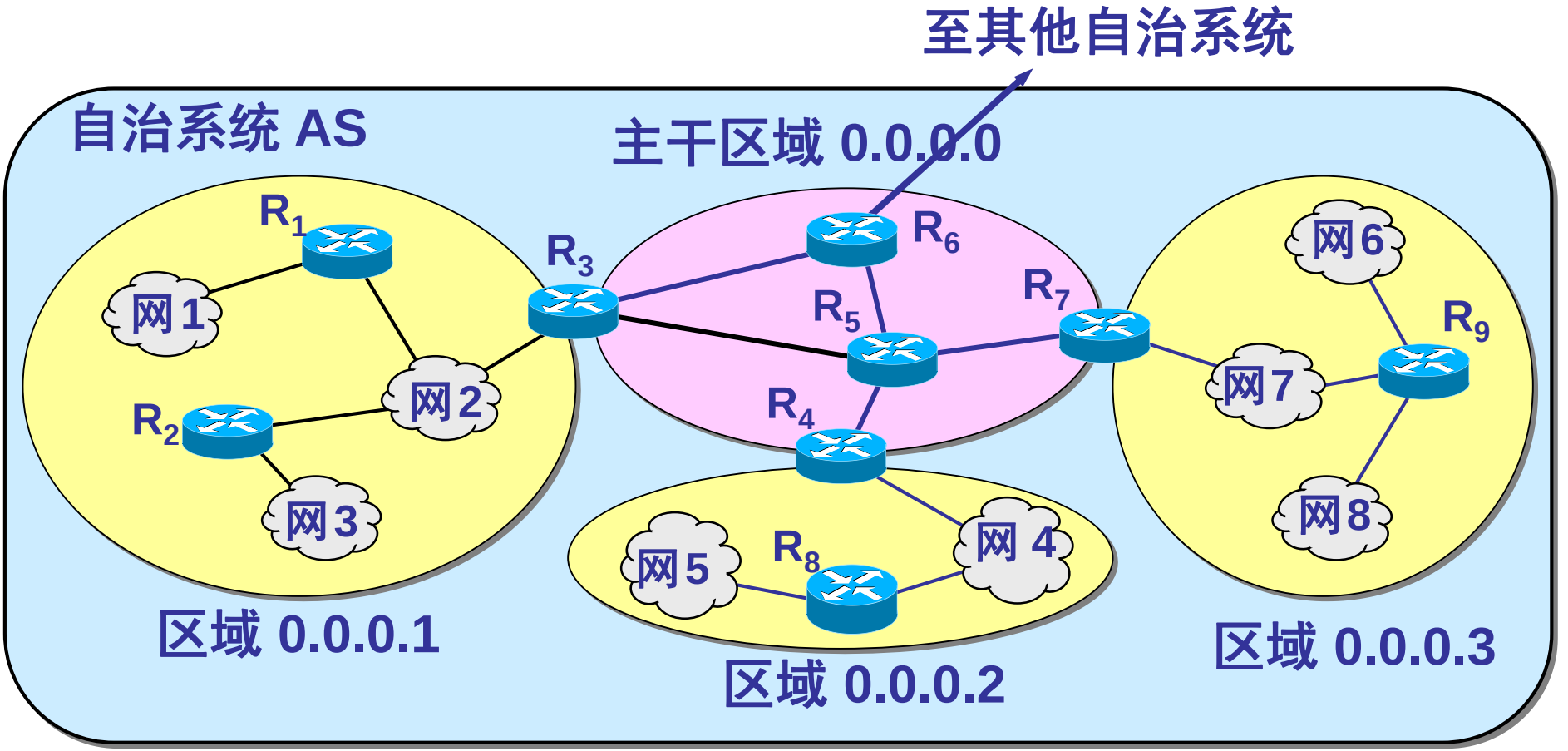
- **和哪些路由器交换信息？**——向本自治系统中所有路由器发送信息，这里使用的方法是**洪泛法**。
- **交换什么信息？**——发送的信息就是与本路由器相邻的所有路由器的**链路状态**，但这只是路由器所知道的部分信息。
  - “链路状态”就是说明本路由器都和哪些路由器相邻，以及该链路的“度量”(metric)。
- **什么时候交换信息？**只有当**链路状态发生变化**时，路由器才用洪泛法向所有路由器发送此信息。

OSPF 的区域(area)



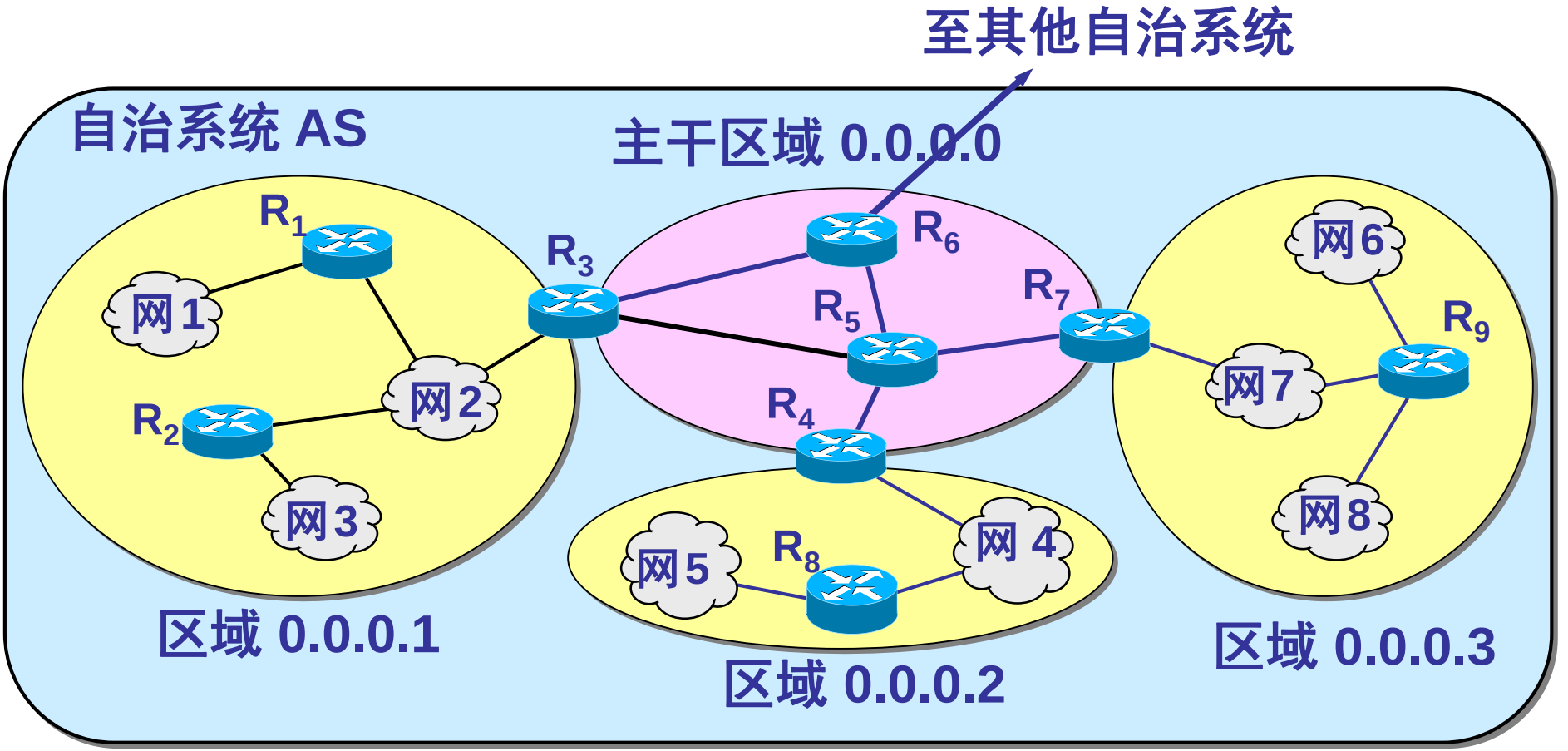
- 为了使 OSPF 能够用于规模很大的网络，OSPF 将一个自治系统再划分为若干个更小的范围，叫作区域。
- 每一个区域都有一个32位的区域标识符（用点分十进制表示）。
- 区域也不能太大，在一个区域内的路由器最好不超过 200 个。

OSPF 的区域(area)



- 将利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个自治系统，减少了整个网络上的通信量。
- 在一个区域内部的路由器只知道本区域的完整网络拓扑，不知道其他区域的网络拓扑的情况。

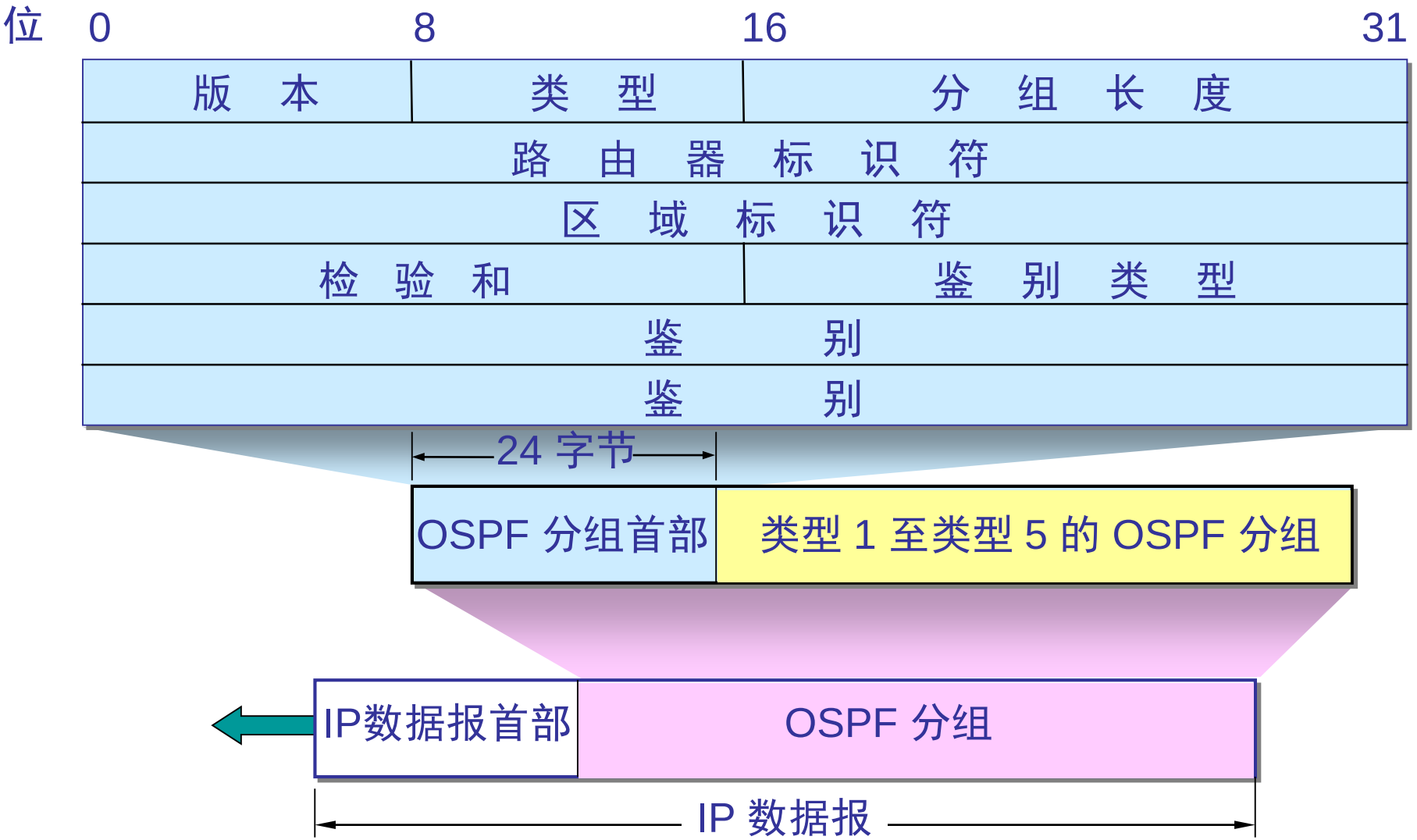
OSPF 的区域(area)



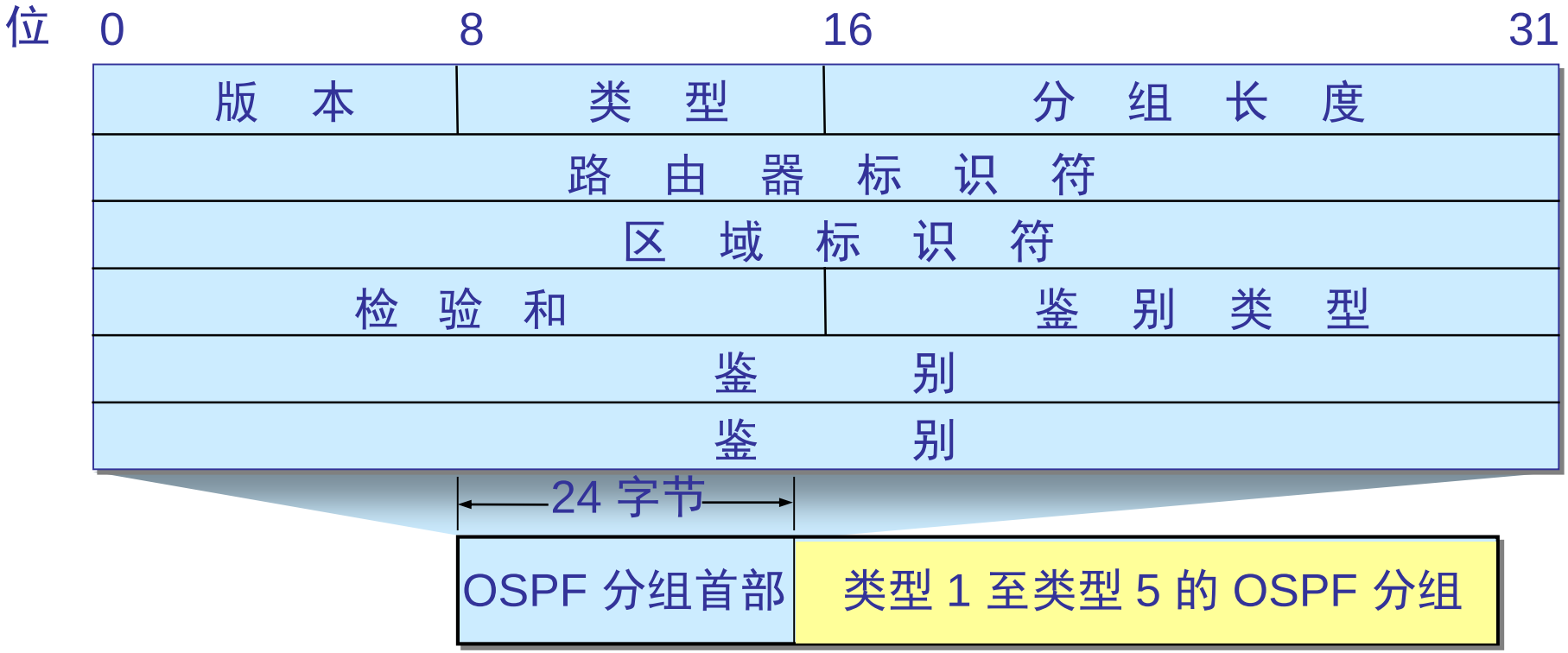
- 采用层次结构划分区域。在上层的区域叫主干区域。主干区域的标识符规定为0.0.0.0，作用是用来连通其他下层区域。
- 主干路由器：R_3，R_4，R_5，R_6，R_7
- 区域边界路由器：R_3，R_4，R_7。自治系统边界路由器：R_6

OSPF 分组（报文）

OSPF分组不用UDP而是直接用IP数据报传送 ：

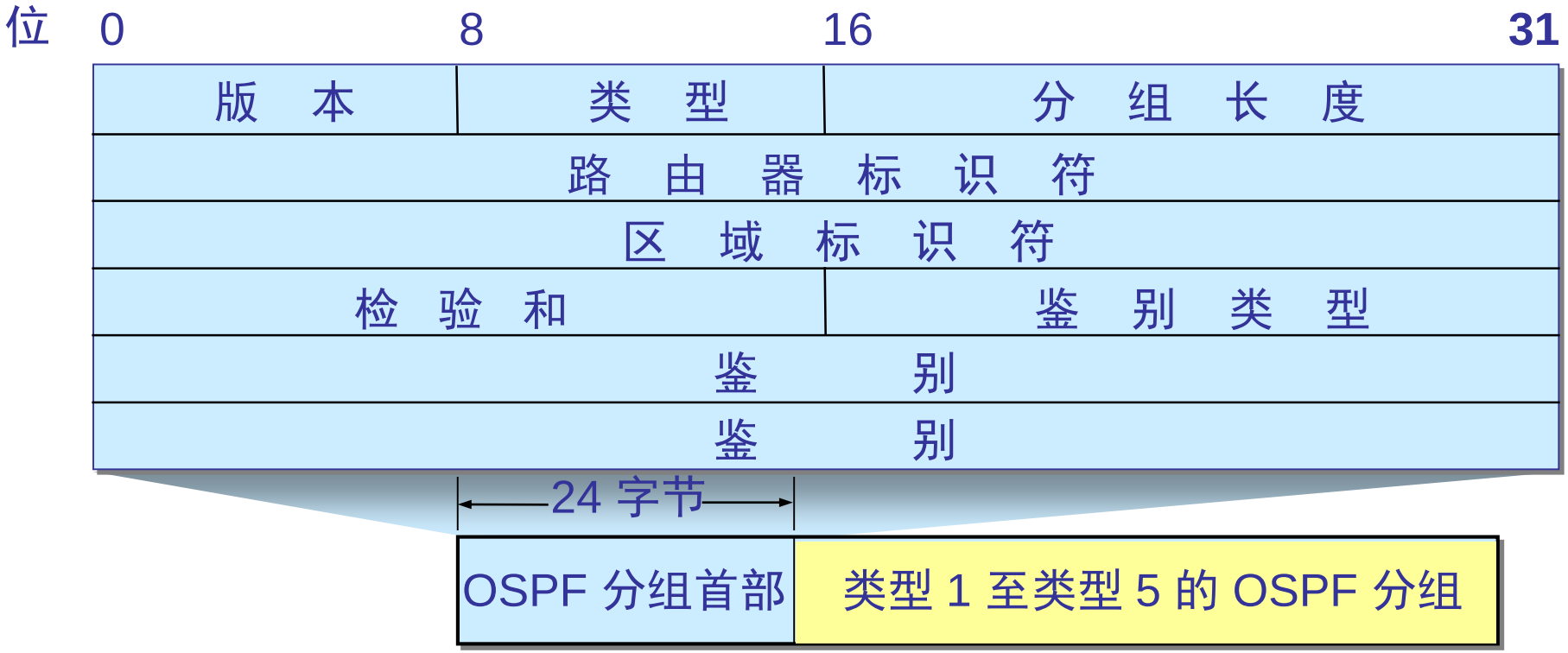


OSPF 分组（报文）



- **版本：** 当前版本号为2
- **类型：** 五种类型分组中的一种
- **分组长度：** 包括首部在内的分组长度
- **路由器标识符：** 发送该分组的路由器接口IP

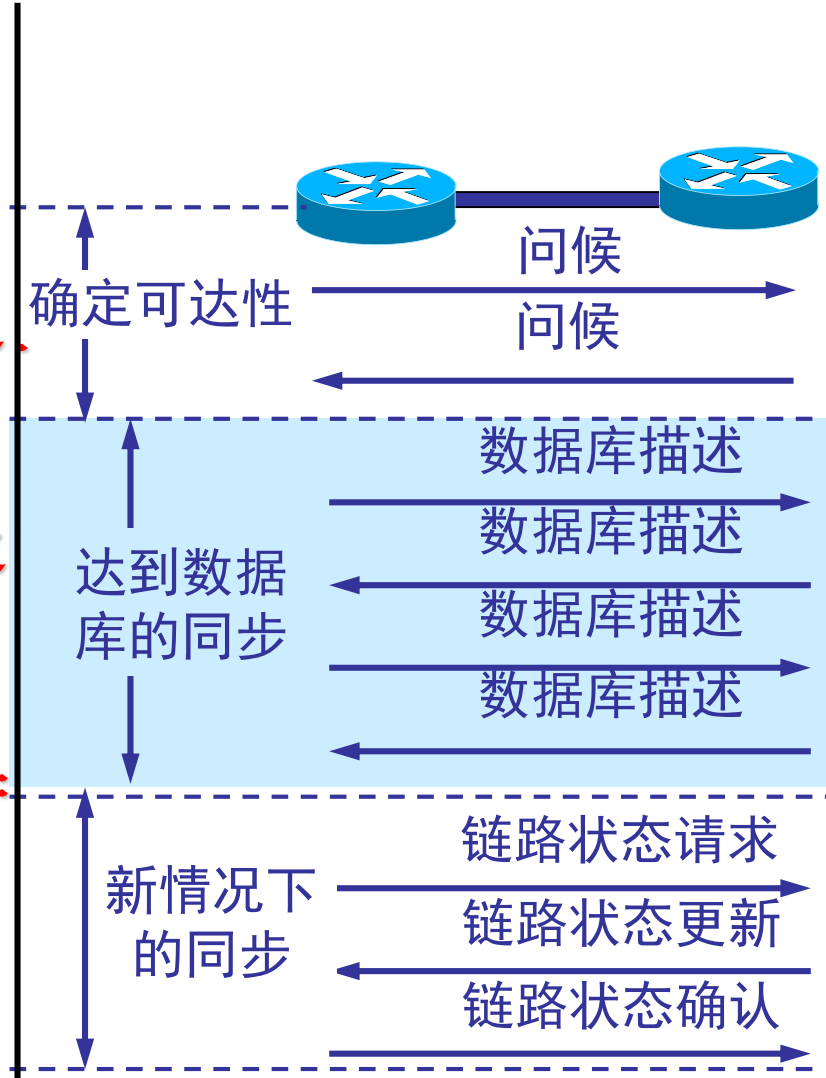
OSPF 分组（报文）



- **区域标识符**：分组所属的区域的标识符
- **校验和**：校验分组中的差错
- **鉴别类型**：目前只有两种：0（不用），1（口令）
- **鉴别**：类型为零时填入0，类型为1时填入8个字符的口令。

OSPF 的五种分组类型

- **问候(Hello)分组**:每隔10秒相邻路由器交换一次问候分组，40秒未收到则认为不可达。
- **数据库描述(Database Description)分组**：给出自己的链路状态数据库中所有项目的摘要信息。
- **链路状态请求(Link State Request)分组**：向对方请求发送某些链路状态项目的详细信息。
- **链路状态更新(Link State Update)分组**：用洪泛法对全网更新链路状态。
- **链路状态确认(Link State Acknowledgment)分组**：对链路状态更新分组的确认。



第一种类型探测可达性，后四种类型用于链路状态数据库同步。

外部网关协议——BGP

边界网关协议

- BGP 是不同自治系统的路由器之间交换路由信息的协议。
- BGP 的较新版本是 1995 年发表的 BGP-4（BGP 的第 4 个版本）。
- 可以将 BGP-4 简写为 BGP。

BGP 使用的环境

- 因特网的规模太大，使得自治系统之间路由选择非常困难。
- 对于自治系统之间的路由选择，要寻找最佳路由是很不现实的。
- 自治系统之间的路由选择必须考虑有关策略。
- 因此，**边界网关协议** BGP 只能是力求寻找一条能够到达目的网络且**比较好的路由**（不能兜圈子），而**并非要寻找一条最佳路由**。

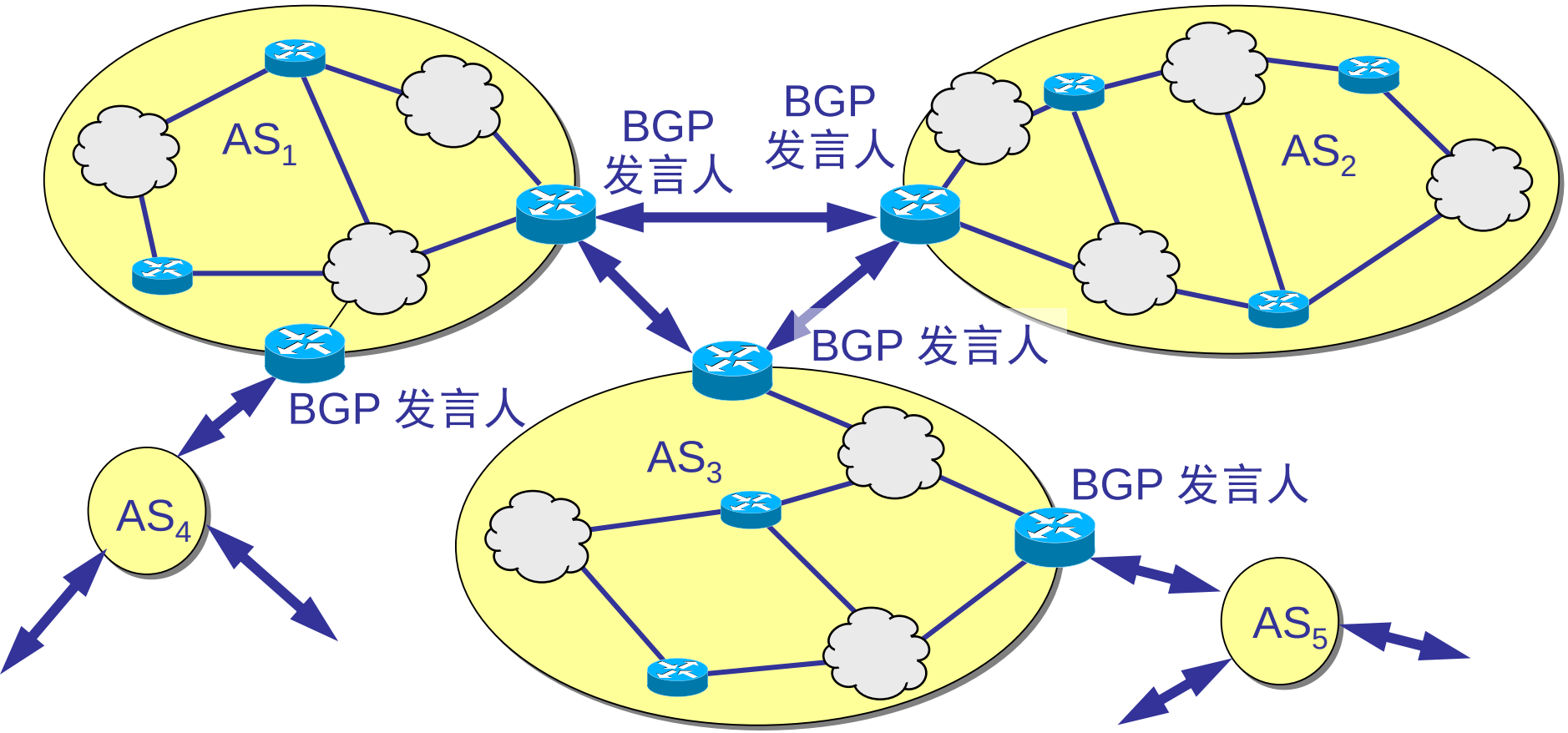
BGP 发言人

- 每一个自治系统的管理员要选择至少一个路由器作为该自治系统的“BGP 发言人”。
- 一般说来，两个 BGP 发言人都是通过一个共享网络连接在一起的，而 BGP 发言人往往就是 BGP 边界路由器，但也可以不是 BGP 边界路由器。

BGP 交换路由信息

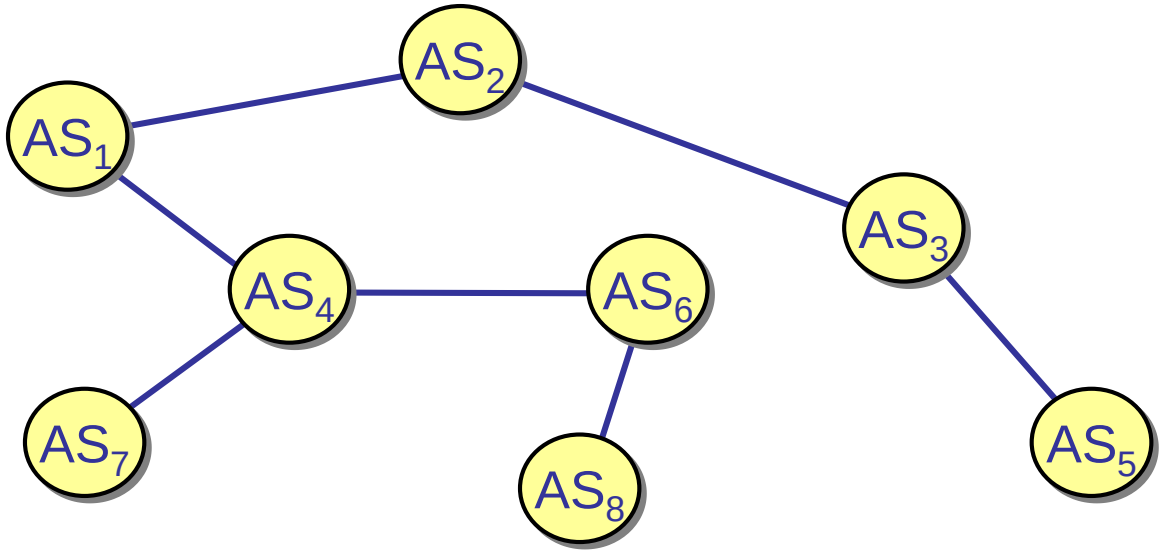
- 一个 BGP 发言人与其他自治系统中的 BGP 发言人要交换路由信息，就要先建立 TCP 连接，然后在此连接上交换 BGP 报文以建立 BGP 会话(session)，利用 BGP 会话交换路由信息。
- 使用 TCP 连接能提供可靠的服务，也简化了路由选择协议。
- 使用 TCP 连接交换路由信息的两个 BGP 发言人，彼此成为对方的邻站或对等站。

BGP 发言人和
自治系统 AS 的关系



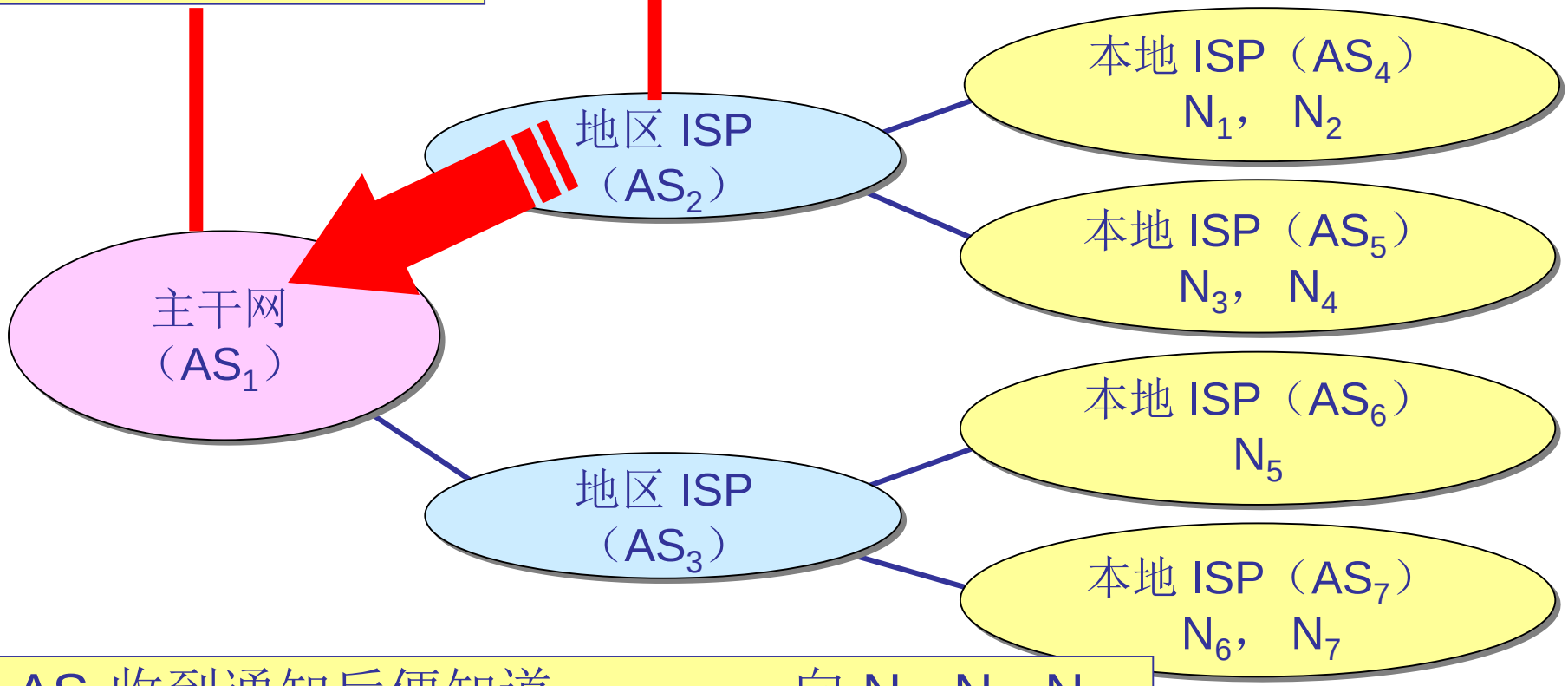
自治系统连通图

- BGP所交换的网络可达性的信息是**到达某个网络所要经过的一系列AS**。
- BGP发言人使用路径**向量选择协议**交换信息。
- BGP 发言人互相交换网络可达性的信息后，各 BGP 发言人就可找出到达各自治系统的比较好的路由。



AS_1通知：“要到达网络 N_1, N_2, N_3 和 N_4 可经过（AS_1，AS_2）”。

AS_2 的 BGP 发言人通知主干网AS1的 BGP 发言人：“要到达网络 N_1, N_2, N_3 和 N_4 可经过 AS_2。”



AS_3收到通知后便知道：N_5, N_6, N_7 向 N_1, N_2, N_3 和 N_4 发送数据时可经过（AS_1，AS_2）”。

BGP 协议的特点

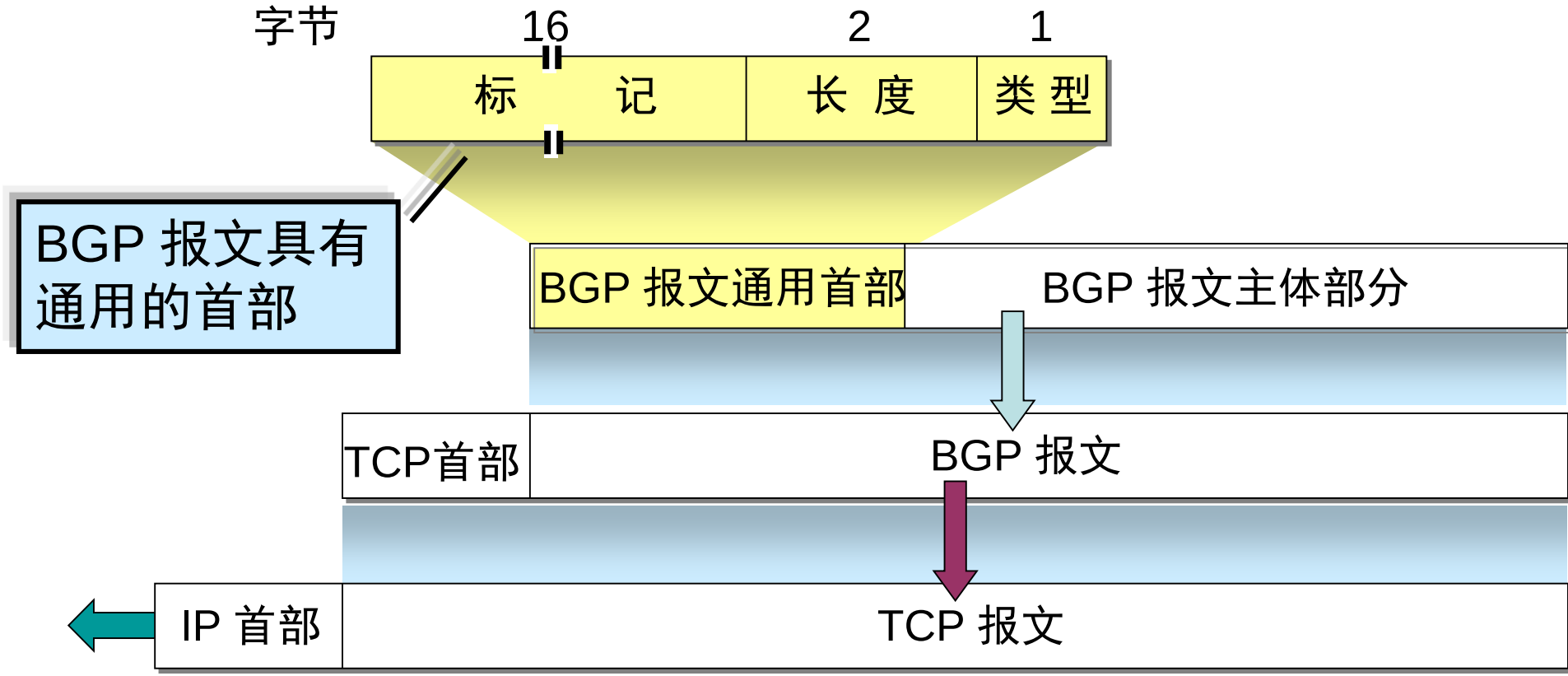
- BGP 协议交换路由信息的结点数量级是自治系统数的量级，这要比这些自治系统中的网络数少很多。
- 每一个自治系统中 BGP 发言人（或边界路由器）的数目是很少的。这样就使得自治系统之间的路由选择不致过分复杂。

BGP 协议的特点

- BGP **支持** CIDR。因此 BGP 的路由表也就应当包括目的网络前缀、下一跳路由器，以及到达该目的网络所要经过的各个自治系统序列。
- 在BGP 刚刚运行时，BGP 的邻站是交换整个的 BGP 路由表。但**以后只需要在发生变化时更新有变化的部分**。这样做对节省网络带宽和减少路由器的处理开销方面都有好处。

BGP 报文

- BGP通过TCP协议交换路由信息。



标记： 鉴别收到的BGP报文。不用时置为全1

长度： 以字节为单位包括首部的BGP报文长度。取值：19～4096.

类型： 1-4，分别对应一种BGP报文。

BGP-4报文的四种类型

(1) **打开(OPEN)报文：**用来与相邻的另一个BGP发言人建立关系。如果邻站接受这种邻站关系，以KEEPALIVE报文响应。

(2) **更新(UPDATE)报文：**用来通告并增加新路由，以及列出要撤消的多条路由。

(3) **保活(KEEPALIVE)报文：**用来确认建立连接和周期性地证实邻站关系（通常间隔30秒）。

(4) **通知(NOTIFICATION)报文：**用来发送检测到的差错。

- 在 RFC 2918 中增加了 ROUTE-REFRESH 报文，用来请求对等端重新通告。 报文只有4字节，不采用上述报文的报文格式。

END